
SIGNALREPORT
NETWORK DIAGNOSTIC TOOL

SignalReport: Internet-Qualitäts-Monitoring

Projektdokumentation eines Open-Source-Projekts.
Hervorgegangen aus dem Abschlussprojekt des Java-Diplomlehrgangs
”Software Developer:in” am WIFI Wien (Kurs 18195015, 2025/2026)
und seither aktiv weiterentwickelt.

https://github.com/matthili/SignalReport

Matthias F.

Jänner 2026 – laufend

Inhaltsverzeichnis

1 Einleitung **6**
- 1.1 Projektziel . . . 6
- 1.2 Problemstellung . . . 6
- 1.3 Projektgeschichte . . . 6
- 1.4 Projektumfang . . . 6

2 Anforderungen **7**
- 2.1 Funktionale Anforderungen . . . 7
- 2.2 Nicht-funktionale Anforderungen . . . 8

3 User Stories **8**
- 3.1 Übersicht der Rollen . . . 8
- 3.2 Kernfunktionen (Must-Have) . . . 8
- 3.3 Erweiterte Funktionen (Should-Have) . . . 9
- 3.4 Systemfunktionen (automatisiert) . . . 10

4 Feature Requests (geplante Erweiterungen) **10**
- 4.1 FR-01: Automatischer PDF-Versand per E-Mail . . . 11
- 4.2 FR-02: Prometheus-Exporter für Grafana-Integration . . . 11
- 4.3 FR-03: Multi-Host-Vergleich . . . 11
- 4.4 FR-04: Docker-Container . . . 11
- 4.5 FR-05: Traceroute-Analyse . . . 12
- 4.6 FR-06: Speedtest-Integration . . . 12
- 4.7 FR-07: Verbesserte Verschlüsselung . . . 12
- 4.8 FR-08: Messenger-Benachrichtigungen bei Ausfällen . . . 13
- 4.9 FR-09: ResultSet-basierte Datenbank-Iteratoren . . . 13
- 4.10 FR-10: Home Assistant Add-on . . . 13
- 4.11 FR-11: Dienst-Erreichbarkeit (Sperr- und Zensur-Erkennung) . . . 14
- 4.12 Zusammenfassung der Feature Requests . . . 15
- 4.13 Bereits umgesetzte Erweiterungen . . . 15

5 Architektur **15**
- 5.1 Gesamtarchitektur . . . 15
- 5.2 Komponenten-Design . . . 16
- 5.3 Klassen-Design . . . 17
- 5.4 Design-Entscheidungen . . . 18
- 5.5 Twin-Datenbank-Architektur . . . 19
- 5.6 Daten-Aufbereitung (Stundenwerte und Aufbewahrung) . . . 20
- 5.7 Mitgelieferte Listen und eigene Einstellungen . . . 21
- 5.8 Internationalisierung (i18n) . . . 22
- 5.9 Datenfluss bei der Messung . . . 23
- 5.10 PDF-Export-Prozess . . . 24
- 5.11 Use-Cases . . . 24
- 5.12 Authentifizierungs-Zustände . . . 24

6 Implementierung **25**
6.1 Mess-Engine und Strategy-Pattern . . . 25
6.2 IP-Tracking und Host-Identifikation . . . 26
6.3 Statistische Auswertung . . . 26
6.4 Web-Interface und Rendering-Architektur . . . 26
6.4.1 Dark Mode . . . 27
6.4.2 DNS-Benchmark . . . 27
6.4.3 Challenge-Response Authentifizierung . . . 28
6.5 PDF-Export . . . 28
6.6 Konfigurationsmanagement . . . 29
6.7 Datenbank-Schema . . . 29

7 Test **30**
7.1 Teststrategie . . . 30
7.2 JUnit-Testabdeckung . . . 31
7.3 Beispiel-Test für Statistik-Berechnung . . . 33
7.4 Testergebnisse . . . 34
7.5 Testumgebung . . . 34
7.6 Test-Philosophie und -Qualität . . . 34

8 Fazit und Ausblick **35**
8.1 Zusammenfassung . . . 35
8.2 Gelerntes . . . 35
8.3 Herausforderungen und Lösungen . . . 36
8.4 Mögliche Erweiterungen . . . 36
8.5 Fazit . . . 36
8.6 Weiterentwicklung nach der Diplomprüfung . . . 37

9 Installation und Betrieb **37**
9.1 Voraussetzungen . . . 37
9.2 Schnellstart (portabel) . . . 37
9.3 Installation als Windows-Dienst . . . 38
9.3.1 Ablauf . . . 38
9.3.2 Verwaltung . . . 38
9.4 Installation unter Linux (systemd) . . . 38
9.4.1 Ablauf . . . 38
9.4.2 Verwaltung . . . 39
9.5 Installation unter macOS (launchd) . . . 39
9.5.1 Unterschiede zu Linux . . . 39
9.5.2 Verwaltung . . . 39
9.6 Synology NAS (Docker) . . . 40
9.7 Konfiguration . . . 40
9.8 Backup und Migration . . . 40

10 REST-API-Referenz **41**
10.1 Authentifizierung . . . 41
10.2 Messdaten . . . 41
10.3 Verfügbarkeit und Störungs-Lokalisierung . . . 42
10.4 Dienst-Erreichbarkeit . . . 42

10.5 Berichte und Export 42
10.6 Daten-Aufbereitung 42
10.7 DNS-Benchmark 42
10.8 IP-Tracking und Host-Info 43
10.9 Konfiguration 43
10.10 Authentifizierung und Setup 43
10.11 Push-Benachrichtigungen 43
10.12 HTTP-Statuscodes 43

11 Anhang A: UML-Diagramme (vollständig) 44
11.1 Klassen-Diagramm 44
11.2 Komponenten-Diagramm 45
11.3 Sequenz-Diagramm: Messungsablauf 46
11.4 Sequenz-Diagramm: PDF-Export 48
11.5 Use-Case-Diagramm 48
11.6 Deployment-Diagramm 49
11.7 State-Diagramm: Authentifizierung 49

1 Einleitung

1.1 Projektziel

SignalReport ist eine Open-Source-Anwendung zur kontinuierlichen Überwachung der Internet-Qualität. Im Gegensatz zu kommerziellen Tools ist es:

- **Kostenlos** und Open Source
- **Einfach zu installieren** (kein Server nötig)
- **Plattformunabhängig** (Windows, macOS, Linux, NAS, Docker)
- **Datensicherheit** (alle Daten lokal gespeichert)

1.2 Problemstellung

Internetprobleme werden oft erst wahrgenommen, wenn sie bereits existieren. SignalReport ermöglicht:

- Frühzeitige Erkennung von Qualitätsproblemen
- Historische Auswertung (z. B. "Warum ist das Netz immer um 19 Uhr langsam?")
- Nachweis für Internetanbieter bei Problemen
- Langfristige Dokumentation für Provider-Beschwerden

1.3 Projektgeschichte

SignalReport entstand zwischen Jänner und April 2026 als Abschlussprojekt des Diplomlehrgangs „Software Developer:in“ (Kurs 18195015) am WIFI Wien. Die ursprüngliche Version wurde im April 2026 erfolgreich präsentiert und bewertet. Seither wird das Projekt als persönliches Open-Source-Vorhaben aktiv weiterentwickelt – mit zusätzlichen Funktionen, robusteren Architekturentscheidungen und kontinuierlicher Pflege.

Diese Dokumentation beschreibt den **aktuellen Entwicklungsstand**, nicht den exakten Stand zum Zeitpunkt der Diplomprüfung. Wer den ursprünglichen, geprüften Stand begutachten möchte, findet ihn als Release `V1` im GitHub-Repository unter https://github.com/matthili/SignalReport/releases.

1.4 Projektumfang

Der aktuelle Stand umfasst:

- Java-Anwendung mit Javalin (Web-UI) und embedded H2-Twin-Datenbank
- Mess-Engines für Ping, DNS und HTTP (plattformspezifisches Ping)
- Statistische Auswertung (95. Perzentil, Jitter, Paketverlust)
- Live-Visualisierung mit Chart.js und Stunden-Heatmap

- PDF-Export (24h / 7 Tage / 12 Monate) und CSV-Export
- IP-Tracking für dynamische IP-Adressen
- Konfigurierbare Messziele und Maintenance-Fenster
- Challenge-Response-Authentifizierung (SHA-256, ohne HTTPS-Pflicht)
- Web-basierter Setup-Wizard (keine CLI nötig)
- Crash-resistente Datenpersistenz durch Twin-Datenbank-Architektur (eingeführt nach Diplomprüfung)
- Mehrsprachigkeit: Web-UI, PDF und CSV in neun Sprachen (eingeführt nach Diplomprüfung)

2 Anforderungen

2.1 Funktionale Anforderungen

SignalReport muss folgende funktionale Anforderungen erfüllen:

- **Kontinuierliche Messung**: Alle X Sekunden Ping, DNS und HTTP messen (konfigurierbar)
- **Web-Interface**: Messungen im Browser anzeigen mit Live-Updates
- **PDF-Export**: Berichte als PDF generieren (24h, 7 Tage, 12 Monate)
- **CSV-Export**: Alle Messdaten als CSV exportieren
- **Statistische Auswertung**: 95. Perzentil, Jitter, Paketverlust berechnen
- **DNS-Benchmark**: Mehrere DNS-Server weltweit vergleichen
- **IP-Tracking**: Externe IP-Änderungen erkennen und dokumentieren
- **IPv4/IPv6**: SignalReport unterstützt Dual-Stack-Messungen (IPv4 + IPv6). Bei Providern mit DS-Lite (Dual-Stack Lite) wie Magenta-Austria kann die externe IPv6-Adresse nicht ermittelt werden, da keine echte IPv6-Route zum Internet existiert. In diesem Fall wird 'unknown' angezeigt – ein technisch korrektes Verhalten gemäß RFC 8415. Die IPv4-Messung bleibt davon unberührt und funktioniert weiterhin zuverlässig.
- **Maintenance-Fenster**: Messungen in definiertem Zeitfenster pausieren (um definierte Wartungsfenster/Neustarts zu kompensieren)
- **Konfiguration**: Messziele, Intervall und Benutzer-Info per UI ändern
- **Authentifizierung**: Optionale Passwort-Sicherung des Web-Interface
- **Setup-Wizard**: Erstmalige Einrichtung über Web-Oberfläche

2.2 Nicht-funktionale Anforderungen

- **Plattformunabhängigkeit**: Lauffähig auf Windows, macOS, Linux, NAS
- **Einfache Installation**: Kein separater Server nötig, portable JAR-Datei
- **Datensicherheit**: Alle Daten lokal gespeichert, keine Cloud-Abhängigkeit
- **Performance**: Messungen alle 5-60 Sekunden ohne Systemlast
- **Benutzerfreundlichkeit**: Intuitive Web-Oberfläche, ohne technisches Vorwissen bedienbar
- **Wartbarkeit**: Sauberer, strukturierter, dokumentierter Code mit JUnit-Tests

3 User Stories

User Stories beschreiben die Anforderungen aus Sicht der Endbenutzer. Sie folgen dem Format *"Als [Rolle] möchte ich [Funktion], damit [Nutzen]."* und helfen dabei, den Fokus auf den tatsächlichen Mehrwert für den Anwender zu legen.

3.1 Übersicht der Rollen

Benutzer Privatperson oder Techniker, der seine Internetqualität überwachen und bei Problemen fundierte Beschwerden beim Provider einreichen möchte.

Administrator Person, die SignalReport einrichtet und die Sicherheitseinstellungen verwaltet (kann identisch mit dem Benutzer sein).

System SignalReport selbst – führt automatisierte Hintergrundprozesse aus.

3.2 Kernfunktionen (Must-Have)

1. **US-01: Kontinuierliche Messung**
   Als Benutzer möchte ich, dass meine Internetverbindung automatisch alle paar Sekunden gemessen wird, damit ich lückenlos dokumentieren kann, wann Probleme auftreten.

   - Akzeptanzkriterien: Ping-, DNS- und HTTP-Messungen laufen im konfigurierbaren Intervall (5 s – 1 h); Ergebnisse werden persistent in der H2-Datenbank gespeichert.

2. **US-02: Live-Dashboard**
   Als Benutzer möchte ich meine Messergebnisse in Echtzeit im Browser sehen, damit ich sofort erkenne, wenn meine Verbindung schlecht ist.

   - Akzeptanzkriterien: Web-UI zeigt Tabelle der letzten Messungen, Live-Chart (Chart.js) und Statistik-Panel; Daten aktualisieren sich alle 5 Sekunden.

3. **US-03: PDF-Bericht für Provider-Beschwerde**
   Als Benutzer möchte ich einen professionellen PDF-Bericht generieren können, damit ich meinem Provider schwarz auf weiß belegen kann, dass die Verbindung regelmäßig einbricht.

- Akzeptanzkriterien: Berichte für 24 h, 7 Tage und 12 Monate; enthalten Kundendaten, Charts mit Zieländerungsmarkierungen, Top-10 schlechteste Messungen und Verbindungsausfälle.

4. **US-04: CSV-Export**
   Als Benutzer möchte ich die Rohdaten als CSV exportieren können, damit ich sie in Excel oder anderen Tools weiterverarbeiten kann.

   - Akzeptanzkriterien: Vollständiger und gefilterter Export möglich; Semikolon als Trennzeichen (für deutsche Excel-Versionen).

5. **US-05: Statistiken**
   Als Benutzer möchte ich Durchschnittslatenz, 95. Perzentil, Jitter und Paketverlust auf einen Blick sehen, damit ich die Qualität meiner Verbindung beurteilen kann.

   - Akzeptanzkriterien: Berechnung für alle drei Messtypen (PING/DNS/HTTP) über konfigurierbare Zeiträume.

6. **US-06: Setup-Assistent**
   Als Administrator möchte ich beim ersten Start durch einen Assistenten geführt werden, damit ich das System ohne Kommandozeile einrichten kann.

   - Akzeptanzkriterien: Web-basierter Wizard setzt Admin-Passwort und optionale Authentifizierung; läuft nur beim allerersten Start.

3.3 Erweiterte Funktionen (Should-Have)

7. **US-07: DNS-Benchmark**
   Als Benutzer möchte ich verschiedene DNS-Server weltweit vergleichen können, damit ich den schnellsten DNS-Server für meinen Standort finde.

   - Akzeptanzkriterien: Parallele Abfrage von mindestens 10 DNS-Servern aus verschiedenen Regionen; Ergebnisse sortiert nach Latenz; 30-Sekunden-Cooldown zwischen Benchmarks.

8. **US-08: IP-Tracking**
   Als Benutzer möchte ich sehen, wann sich meine externe IP-Adresse ändert, damit ich nachvollziehen kann, wie oft mein Provider die Verbindung trennt.

   - Akzeptanzkriterien: Änderungen werden mit Zeitstempel protokolliert; Anzeige im UI mit alter und neuer IP; Statistik über Anzahl der Änderungen.

9. **US-09: Wartungsfenster**
   Als Benutzer möchte ich ein Zeitfenster definieren können, in dem keine Messungen stattfinden, damit geplante Router-Neustarts die Statistik nicht verfälschen.

   - Akzeptanzkriterien: Start- und Endzeit konfigurierbar; funktioniert auch über Mitternacht (z. B. 23:50 – 00:10); Status im UI sichtbar.

10. **US-10: Dark Mode**
    Als Benutzer möchte ich zwischen hellem und dunklem Design wechseln können, damit die Oberfläche auch bei nächtlicher Nutzung angenehm ist.

- Akzeptanzkriterien: Toggle-Button im Header; Einstellung persistent in config.json; kein Flackern beim Laden (Server-seitige CSS-Klassen-Injection); Chart.js-Farben passen sich an.

11. **US-11: Authentifizierung**
Als Administrator möchte ich den Zugriff auf das Web-Interface mit Passwort schützen können, damit Unbefugte nicht meine Messdaten sehen oder Einstellungen ändern können.

    - Akzeptanzkriterien: Challenge-Response-Authentifizierung (SHA-256) mit Admin- und User-Rolle; Passwort wird nie im Klartext übertragen; Session-basiert mit 24 h Timeout; Admin hat Vollzugriff, User nur Leserechte; kann jederzeit aktiviert/deaktiviert werden.

12. **US-12: Push-Benachrichtigungen**
Als Benutzer möchte ich bei Verbindungsausfällen oder hoher Latenz automatisch benachrichtigt werden, damit ich auch ohne offenes Browser-Fenster informiert bin.

    - Akzeptanzkriterien: Browser Push Notifications; konfigurierbarer LatenzSchwellwert und Anzahl aufeinanderfolgender schlechter Messungen.

3.4 Systemfunktionen (automatisiert)

13. **US-13: Hintergrund-Dienst**
Als System möchte ich als Hintergrund-Dienst laufen (auch ohne angemeldeten Benutzer), damit die Messung rund um die Uhr erfolgt.

    - Akzeptanzkriterien: Installations-Skripte für Windows (procrun), Linux (systemd) und macOS (launchd); automatischer Start nach Neustart.

14. **US-14: Plattformunabhängigkeit**
Als System möchte ich auf Windows, macOS, Linux und Synology NAS lauffähig sein, damit der Benutzer freie Wahl beim Einsatzort hat.

    - Akzeptanzkriterien: Einzelne ausführbare JAR-Datei; embedded H2-Datenbank ohne separaten Server; getestet auf allen vier Plattformen.

4 Feature Requests (geplante Erweiterungen)

Während User Stories den *aktuellen* Funktionsumfang beschreiben, dokumentieren Feature Requests die *zukünftigen* Erweiterungsmöglichkeiten. Sie entstehen aus eigenen Ideen, Rückmeldungen potenzieller Nutzer und technischen Überlegungen.

Die Priorisierung erfolgt nach dem MoSCoW-Schema:

- **Must** – Wird in der nächsten Version umgesetzt
- **Should** – Sollte bald umgesetzt werden
- **Could** – Wäre schön, aber nicht dringend
- **Won't (yet)** – Aktuell nicht geplant, aber vorstellbar

4.1 FR-01: Automatischer PDF-Versand per E-Mail

Priorität: Should

Beschreibung: Der Benutzer soll einen Zeitplan konfigurieren können (z. B. wöchentlich oder monatlich), nach dem SignalReport automatisch einen PDF-Bericht generiert und per SMTP an eine konfigurierte E-Mail-Adresse versendet.

Motivation: Viele Benutzer möchten regelmäßige Berichte erhalten, ohne sich manuell einloggen zu müssen. Besonders für Provider-Beschwerden ist ein monatlicher Report hilfreich.

Technische Überlegungen: SMTP-Konfiguration in config.json; javax.mail oder Jakarta Mail; Scheduler via `ScheduledExecutorService`.

4.2 FR-02: Prometheus-Exporter für Grafana-Integration

Priorität: Could

Beschreibung: Ein `/metrics`-Endpoint im Prometheus-Exposition-Format soll bereitgestellt werden, damit bestehende Monitoring-Infrastrukturen (Grafana, Prometheus) die SignalReport-Daten integrieren können.

Motivation: Fortgeschrittene Benutzer haben oft bereits ein Grafana-Dashboard und möchten die Internetqualität dort einbinden.

Technische Überlegungen: Einfaches Text-Format; keine externe Library nötig. Metriken:

- `signalreport_ping_latency_ms`
- `signalreport_dns_latency_ms`
- `signalreport_http_latency_ms`
- `signalreport_packet_loss_percent`

4.3 FR-03: Multi-Host-Vergleich

Priorität: Could

Beschreibung: Wenn SignalReport auf mehreren Geräten läuft (z. B. Desktop und NAS), sollen die Messdaten in einer zentralen Ansicht verglichen werden können.

Motivation: Ermöglicht die Unterscheidung, ob ein Problem am Gerät (WLAN vs. Kabel) oder am Provider liegt.

Technische Überlegungen: Bereits vorbereitet durch das Host-Hash-System in der Datenbank; erfordert API-Endpunkte für hostübergreifende Abfragen.

4.4 FR-04: Docker-Container

Priorität: Should

Beschreibung: Ein offizielles Docker-Image soll bereitgestellt werden, das SignalReport als Container ausführbar macht (z. B. auf einem NAS mit Docker-Support oder in einer Cloud-Umgebung).

Motivation: Docker ist der De-facto-Standard für einfache Deployments; besonders auf Synology NAS mit Container Manager.

Technische Überlegungen: Dockerfile basierend auf `eclipse-temurin:21-jre`; Volume-Mount für `data/` und `config.json`; Port-Mapping 4567.

4.5 FR-05: Traceroute-Analyse

Status: Umgesetzt (Kernidee) – siehe `network.GatewayDiscovery` und `report.ConnectivityAsses`

Beschreibung: Statt einer vollständigen Pfad-Visualisierung wird per Traceroute die lokale Gateway-Kette ermittelt (nahes = Router, fernes = Internet-Gateway) und daraus eine Störungs-Lokalisierung abgeleitet. Eine vollständige Hop-für-Hop-Visualisierung bleibt offen.

Motivation: Hilft bei der Diagnose, ob das Problem beim eigenen Router, beim Internet-Gateway oder beim Provider liegt.

Umsetzung: `GatewayDiscovery` (Traceroute mit RFC-1918-Filter, Erkennung virtueller Gateways in VM/Docker), `ConnectivityAssessment` (Schuld-Verdikt) sowie der zugehörige UI- und PDF-Abschnitt; plattformabhängig (traceroute/tracert).

4.6 FR-06: Speedtest-Integration

Priorität: Won't (yet)

Beschreibung: Neben Latenz-Messungen soll auch die tatsächliche Bandbreite (Download/Upload) gemessen werden können.

Motivation: Für Provider-Beschwerden ist die Bandbreite oft relevanter als die Latenz. Viele Nutzer erwarten einen „Speedtest“ als Grundfunktion.

Technische Überlegungen: Erfordert definierte Test-Server oder eine API wie Ookla/Cloudflare Speedtest; erzeugt signifikanten Traffic; nicht für kurze Intervalle geeignet.

4.7 FR-07: Verbesserte Verschlüsselung

Priorität: Could

Beschreibung: Derzeit werden Passwörter mit SHA-256 gehasht und über ein Challenge-Response-Verfahren übertragen (kein Klartext). Eine zukünftige Version könnte bcrypt oder Argon2 für stärkeres Passwort-Hashing verwenden. Zusätzlich könnte HTTPS-Unterstützung (über Let's Encrypt oder selbstsignierte Zertifikate) für verschlüsselte Kommunikation ergänzt werden.

Motivation: SHA-256 ist für Challenge-Response geeignet, aber nicht speziell für Passwort-Hashing konzipiert. Moderne Algorithmen wie bcrypt bieten durch einstellbare Kostenfaktoren besseren Brute-Force-Schutz. HTTPS schützt die gesamte Datenübertragung zwischen Browser und Server.

Technische Überlegungen: jBCrypt oder Bouncy-Castle-Library für Argon2; Javalin-SSL-Plugin oder vorgeschalteter Reverse-Proxy (nginx/Caddy) für HTTPS.

4.8 FR-08: Messenger-Benachrichtigungen bei Ausfällen

Priorität: Could

Beschreibung: Nach Behebung eines Internetausfalls oder bei unregelmäßigen Ping-Zeiten könnte SignalReport Benachrichtigungen über Messenger-APIs versenden (z. B. Telegram Bot API, Signal, WhatsApp Business API). Dies geht über die bestehenden Browser-Push-Benachrichtigungen hinaus, da Nutzer auch auf ihren Mobilgeräten erreicht werden, wenn sie nicht am Computer sind.

Motivation: Browser-Push-Benachrichtigungen setzen voraus, dass der Browser geöffnet ist. Messenger-Benachrichtigungen erreichen den Benutzer jederzeit auf dem Smartphone – auch unterwegs oder nachts.

Technische Überlegungen: Telegram Bot API (einfachste Integration, kein Kosten); Signal-CLI oder signal-bot; WhatsApp Business API (kostenpflichtig); konfigurierbarer Webhook-Endpunkt als generische Lösung.

4.9 FR-09: ResultSet-basierte Datenbank-Iteratoren

Priorität: Should

Beschreibung: Bei sehr großen Datenmengen (z. B. 12-Monats-Berichte mit Millionen von Messungen) kann der aktuelle Ansatz, alle Daten in eine `List<Measurement>` zu laden, erheblichen Arbeitsspeicher verbrauchen. Eine zukünftige Optimierung könnte ResultSet-basierte Iteratoren (Cursor-basiertes Streaming) verwenden, um Daten zeilenweise zu verarbeiten, ohne alles gleichzeitig im RAM zu halten. Dies würde insbesondere dem CSV-Export und der PDF-Generierung für lange Zeiträume zugutekommen.

Motivation: Bei einem Messintervall von 10 Sekunden entstehen pro Tag ca. 8.640 Messungen – über 12 Monate sind das über 3 Millionen Datensätze. Deren gleichzeitiges Laden kann auf ressourcenschwachen Geräten (z. B. NAS) zu Speicherengpässen führen.

Technische Überlegungen: H2 unterstützt scrollbare ResultSets; `Statement.setFetchSize()` für Streaming; Iterator-Pattern um `ResultSet` als Wrapper; Änderungen in `H2MeasurementRepository` und `PdfReportGenerator`.

4.10 FR-10: Home Assistant Add-on

Priorität: Could

Beschreibung: SignalReport könnte als offizielles Add-on für Home Assistant bereitgestellt werden. Benutzer könnten es direkt über den Add-on Store installieren, ohne sich mit Java, JAR-Dateien oder Installations-Skripten befassen zu müssen.

Motivation: Home Assistant ist die verbreitetste Open-Source-Plattform für Smart-Home-Automatisierung und läuft häufig auf denselben Geräten, die auch für SignalReport geeignet sind (Raspberry Pi, Mini-PCs, NAS). Ein Add-on würde die Installationsschwelle nochmals deutlich senken – von „JAR herunterladen und Skript ausführen“ zu „ein Klick im Add-on Store“. Zusätzlich könnten die Messdaten in Home-Assistant-Dashboards und Automatisierungen einfließen (z. B. Benachrichtigung bei Ausfall).

Technische Überlegungen: Home Assistant Add-ons basieren auf Docker-Containern; der bereits geplante Docker-Container (FR-04) wäre die Grundlage. Ergänzend: `config.yaml` für die Add-on-Konfiguration, Ingress-Proxy für nahtlose Integration in die Home-Assistant-UI, MQTT- oder REST-Sensor für die Messdaten-Übernahme.

4.11 FR-11: Dienst-Erreichbarkeit (Sperr- und Zensur-Erkennung)

Priorität: Should

Status: **Umgesetzt** (Version 2.0) – siehe `network.ServiceReachabilityProbe`, `report.ServiceReac` und `ServiceReachabilityScheduler`.

Beschreibung: SignalReport prüft zusätzlich, ob ausgewählte Online-Dienste (z. B. Facebook, Instagram, X, YouTube, Wikipedia, WhatsApp) aus der Netzwerk-Situation des Messpunkts erreichbar sind – und unterscheidet dabei, ob ein Dienst *selbst* gestört ist oder ob er *gesperrt* wird (per DNS, TCP, SNI/DPI oder Sperrseite). Diese Checks laufen in einem eigenen, deutlich langsameren Takt (Standard alle 6 h), sind per Knopfdruck mit einer Abkühlphase von 5 min auslösbar und vollständig abschaltbar (Standard: aus).

Motivation: Liefert eine Langzeit-Statistik über die Erreichbarkeit einzelner Plattformen – etwa „ab 1. März nicht mehr erreichbar, am 11. März (nach der Wahl) wie durch ein Wunder wieder“. Bewusst getrennt von der Verbindungsqualität: eine kurze Sperre ist belanglos, der langfristige Trend ist die eigentliche Aussage.

Technische Überlegungen: Eigener Daemon-Thread, getrennt vom 30-Sekunden-Messtakt. Pro Dienst eine Schicht-Probe (DNS über ISP- vs. öffentlichen Resolver, TCP-Connect, TLS mit echtem vs. Kontroll-SNI, HTTP-Status/451/Sperrseiten-Fingerprint), deren reine Klassifizierung – analog zu `ConnectivityAssessment` – ein Verdikt liefert (erreichbar / Dienst-Störung / DNS- bzw. TCP- bzw. SNI-Sperre / Sperrseite). Ein *Leitungs-Gate* nutzt den laufenden 30-Sekunden-Monitor: steht die Internet-Verbindung gerade nicht, wird der Lauf übersprungen und ein neutraler Marker geschrieben (analog zum Wartungs-Marker), damit ein Leitungsausfall nicht als Dienst-Sperre fehlinterpretiert wird. Speicherung in einer eigenen Tabelle `service_checks` (Twin-DB), Verdichtung aufeinanderfolgender Checks zu Episoden, Ampel-/Kachel-Anzeige je Dienst (im Stil der DNS-Benchmark-Kacheln), optionaler PDF-Abschnitt und i18n in allen neun Sprachen. Messenger (WhatsApp, Telegram, Signal) erlauben nur eine gröbere Endpunkt-Erreichbarkeit; Mastodon ist föderiert (geprüft wird eine konkrete Instanz).

4.12 Zusammenfassung der Feature Requests

ID	Feature	Priorität
FR-01	Automatischer PDF-Versand per E-Mail	Should
FR-02	Prometheus-Exporter (Grafana)	Could
FR-03	Multi-Host-Vergleich	Could
FR-04	Docker-Container	Should
FR-05	Traceroute-Analyse (Störungs-Lokalisierung)	**Umgesetzt**
FR-06	Speedtest-Integration	Won't (yet)
FR-07	Verbesserte Verschlüsselung	Could
FR-08	Messenger-Benachrichtigungen bei Ausfällen	Could
FR-09	ResultSet-basierte Datenbank-Iteratoren	Should
FR-10	Home Assistant Add-on	Could
FR-11	Dienst-Erreichbarkeit (Sperr-/Zensur-Erkennung)	**Umgesetzt**

Tabelle 1: Feature Requests mit MoSCoW-Priorisierung

4.13 Bereits umgesetzte Erweiterungen

Über die obige Wunschliste hinaus wurden seit der Diplomprüfung mehrere Erweiterungen umgesetzt, die nicht Teil des ursprünglichen FR-Katalogs waren:

- **Dienst-Erreichbarkeit / Sperr-Erkennung** (FR-11, Version 2.0): Schicht-Probe (DNS über ISP- vs. öffentlichen Resolver, TCP, TLS mit echtem vs. Kontroll-SNI, HTTP) je Dienst mit Leitungs-Gate, Episoden-Verdichtung in der Tabelle `service_checks`, Ampel-/Kachel-UI und optionalem PDF-Abschnitt – standardmäßig deaktiviert, in allen neun Sprachen.
- **Störungs-Lokalisierung** (FR-05, Kernidee): Gateway-Kette per Traceroute, Schuld-Verdikt (Router / Internet-Gateway / Internet) sowie Erkennung virtueller Gateways in VM/Docker.
- **Verfügbarkeit & Zuverlässigkeit**: lückenbewusste Kennzahlen (Uptime, Abdeckung, MTBF, MTTR) und aggregierte Verbindungsausfälle, einzeln aus der Wertung nehmbar (DB-Spalte `excluded`).
- **Gateway-Einstellungen**: manuelle Router-/Gateway-IP, das Abschalten des kontinuierlichen Pings für das Internet-Gateway sowie persistent-vs-neu-ermitteln bei lokalem IP-Wechsel.
- **Mess-Tabelle pro Messzyklus** (aufklappbar) sowie ein Refactoring in eine geschichtete 10-Paket-Struktur mit ausgelagerten statischen Web-Assets (`app.css`/`app.js`).

5 Architektur

5.1 Gesamtarchitektur

SignalReport folgt einer monolithischen Architektur, bei der alle Komponenten in einer einzigen JAR-Datei zusammengefasst sind. Diese Entscheidung wurde bewusst getroffen,

um die Installation so einfach wie möglich zu halten – es wird kein separater Datenbankserver, kein Application-Server und kein Build-System beim Endbenutzer benötigt.

Das Deployment-Diagramm in Abbildung 1 zeigt die drei Ebenen der Architektur: den Client (Browser), das Server-System (JVM mit allen Komponenten) und die externen Dienste (DNS-Server, Web-Server, ipify.org).

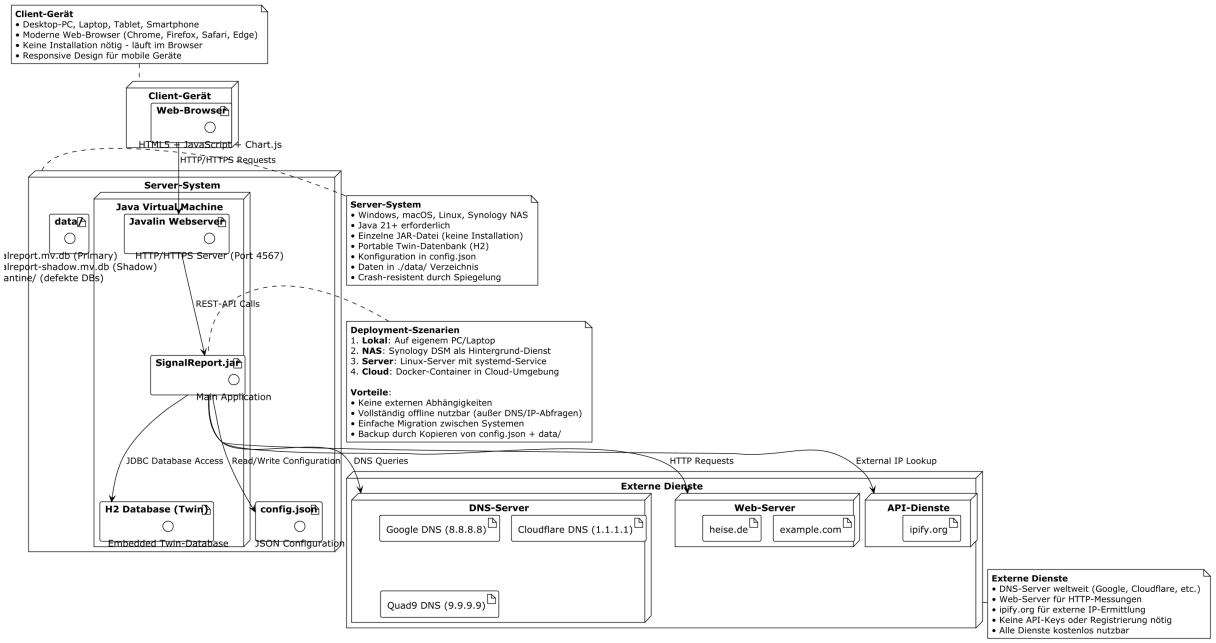


Abbildung 1: Deployment-Diagramm von SignalReport

5.2 Komponenten-Design

Die Anwendung gliedert sich in zehn geschichtete Packages unterhalb von `at.mafue.signalreport` (Abbildung 2):

config Schlanke `Config`-Fassade (JSON-Persistenz, Passwort-Hashing, Defaults) sowie je eine eigene Datei pro Konfigurationsbereich (`MeasurementConfig`, `Targets`, `GatewayConfig`, `MaintenanceWindow`, `AuthConfig`, `ThemeConfig`, …).

measurement Mess-Engine mit dem `Measurer`-Interface und drei Implementierungen (Ping, DNS, HTTP), dem DNS-Benchmark sowie dem zentralen Transferobjekt `Measurement`.

network `GatewayDiscovery` (Traceroute-Gateway-Kette nah/fern, virtuelle-Gateway-Erkennung), `NetworkInfo` (lokale/externe IP) und `HostIdentifier` (stabiler Host-Hash) sowie `ServiceReachabilityProbe` (Schicht-Probe DNS/TCP/TLS-SNI/HTTP für die Dienst-Erreichbarkeit).

storage `H2MeasurementRepository` für die embedded Datenbank in Twin-Architektur (Primary + Shadow) samt eigenen Lese-DTOs (`Statistics`, `IpChange`, `IpChangeStats`, `HourlyAverage`, `HostInfo`). Tabellen: Messungen, Hosts, IP-Änderungen, Dienst-Prüfungen (`ServiceCheck`). Siehe Abschnitt 5.5.

report `ReliabilityReport` (lückenbewusste Kennzahlen), `ConnectivityAssessment` (Schuld-Verdikt), `ServiceReachabilityAssessment/Report` (Dienst-Erreichbarkeit) und `PdfReportGenerator` (OpenPDF + JFreeChart).

web `WebServer` als Javalin-Orchestrator (Setup, Setup-/Auth-Gating-Filter, Registrierung der Routen), `SessionManager` und `ErrorResponse`.

web.view HTML-Renderer `HtmlPageRenderer`, `SetupPageRenderer` und `LoginPageRenderer`.

web.api Zehn Routen-Registrar-Klassen (`PageRoutes`, `MeasurementRoutes`, `ReliabilityRoutes`, …) mit je einer statischen `register(app, …)`-Methode.

i18n `I18n` für die Mehrsprachigkeit (9 Sprachen).

notification `PushNotificationService` für Browser-Benachrichtigungen.

Den Einstiegspunkt bildet `SignalReportApp` mit der kontinuierlichen Mess-Schleife. Das CSS und JavaScript der Oberfläche sind in statische Ressourcen (`src/main/resources/web/app.cs` und `app.js`) ausgelagert.

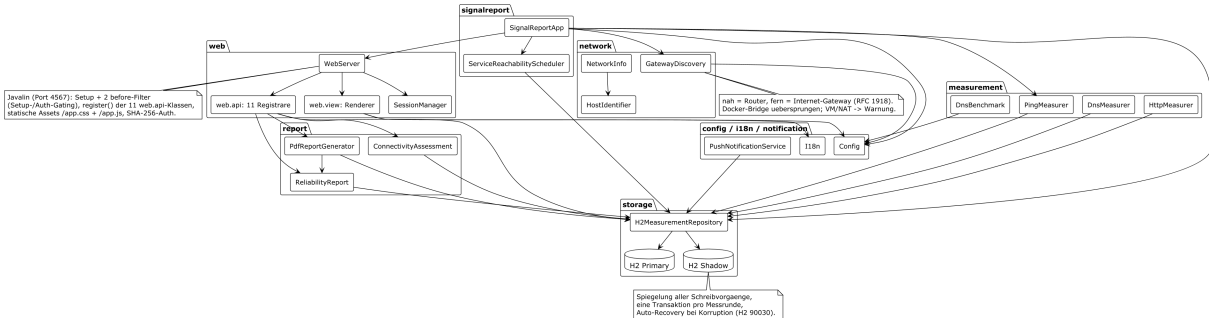


Abbildung 2: Komponenten-Übersicht von SignalReport

5.3 Klassen-Design

Das Klassen-Diagramm in Abbildung 3 zeigt die vollständige Klassenstruktur. Besonders hervorzuheben sind:

- Das **Measurer-Interface** (Strategy-Pattern): Alle drei Messtypen implementieren das gleiche Interface `Measurer` mit der Methode `measure(String target)`. Dadurch können neue Messtypen hinzugefügt werden, ohne bestehenden Code zu ändern (Open/Closed Principle).
- Die **Renderer- und Routen-Aufteilung**: Das HTML-Rendering wurde in eigenständige Klassen ausgelagert (Paket `web.view`), und die Routen wurden nach Domäne in zehn Registrar-Klassen (Paket `web.api`) zerlegt. Der `WebServer` schrumpfte dadurch auf einen reinen Orchestrator von ca. 140 Zeilen; zusätzlich wurden CSS und JavaScript in statische Ressourcen (`app.css`/`app.js`) ausgelagert.
- Die **aufgeteilte Konfiguration**: Statt verschachtelter Klassen liegt jeder Konfigurationsbereich in einer eigenen Datei im Paket `config`; die zentrale `hashPassword()`-Methode vermeidet duplizierten Code.

- Die **Störungs-Lokalisierung**: `GatewayDiscovery` und `ConnectivityAssessment` bestimmen, ob Router, Internet-Gateway oder Provider für eine Störung verantwortlich ist; `ReliabilityReport` liefert lückenbewusste Verfügbarkeits-Kennzahlen (Uptime, Abdeckung, MTBF, MTTR).
- Die **Dienst-Erreichbarkeit** (Version 2.0): `ServiceReachabilityProbe` prüft per Schicht-Probe (DNS über ISP- vs. öffentlichen Resolver, TCP, TLS mit echtem vs. Kontroll-SNI, HTTP), ob ausgewählte Online-Dienste erreichbar oder gesperrt sind; `ServiceReachabilityAssessment` liefert das Verdikt, `ServiceReachabilityReport` verdichtet es zu Episoden. Ein Leitungs-Gate verhindert Fehlalarme bei Internet-Ausfällen.

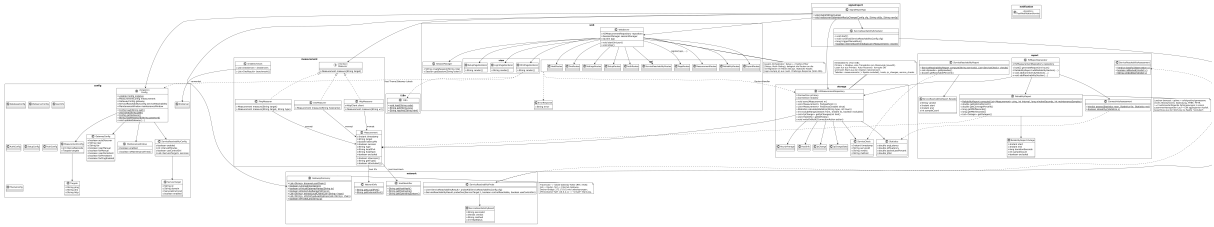


Abbildung 3: Klassen-Diagramm von SignalReport

5.4 Design-Entscheidungen

Entscheidung	**Begründung**
Monolithische Architektur	Einfachste Installation: eine JAR-Datei, keine externen Abhängigkeiten
H2 Embedded Database	Plattformunabhängig, kein separater DB-Server, portable Datei
Twin-DB-Architektur	Synchrone Spiegelung in zweite H2-Datei; bei Korruption wird die intakte Kopie automatisch zur Reparatur verwendet (vgl. Abschnitt 5.5)
Javalin statt Spring Boot	Minimaler Overhead (ca. 5 MB vs. 30+ MB), schneller Startup
Java 21	Virtual Threads, Pattern Matching, Text Blocks fuer HTML-Templates
OpenPDF statt PDFBox	Einfachere API für Tabellen und Charts; bessere Schriftarten-Unterstützung
Inline-HTML statt Templates	Kein Template-Engine-Overhead; Java Text Blocks ermöglichen lesbaren HTML-Code; Dark Mode via CSS Custom Properties
Apache Commons Daemon	Professioneller Dienst-Betrieb auf allen Plattformen (procrun/jsvc)

Tabelle 2: Architektur-Entscheidungen (Architecture Decision Records)

5.5 Twin-Datenbank-Architektur

Nach der Diplomprüfung trat im Produktivbetrieb ein Korruptionsfall der H2-Datenbank auf, ausgelöst durch einen unvermittelten Windows-Update-Neustart genau während eines Schreibvorgangs. Die `.mv.db`-Datei war anschließend nicht mehr lesbar, und auch das offizielle H2-Recovery-Tool konnte keinen einzigen konsistenten Chunk mehr finden. Die historischen Messdaten waren damit unwiederbringlich verloren.

Daraufhin wurde die Persistenzschicht von einer einzelnen H2-Datenbank auf eine **Twin-Architektur** mit zwei parallel geführten H2-Dateien umgestellt:

- **Primary** (`signalreport.mv.db`): autoritative Quelle für alle Lese-Operationen.
- **Shadow** (`signalreport-shadow.mv.db`): synchrone Spiegelung aller Schreiboperationen.

Jede Schreiboperation (`save`, `registerHost`, `recordIpChange`) wird über einen zentralen `writeOnBoth(…)`-Helper sequenziell auf beide Connections angewendet – zuerst auf die Primary, danach auf die Shadow. Lese-Operationen gehen ausschließlich auf die Primary, was den Performance-Overhead minimiert.

Schutz-Stufen. Vier zusammenwirkende Mechanismen sorgen für Robustheit gegen abrupte Prozess-Terminierungen (Stromausfall, Update-Neustart, kill -9) und gegen schleichende Dateischäden:

1. **Eine Transaktion pro Messrunde** (`saveAll`) bei H2-Standard-`WRITE_DELAY` (500 ms): H2 bündelt die Commits und kompaktiert die Datei laufend im Hintergrund. Bis Version 2.0.1 stand `WRITE_DELAY=0` im JDBC-URL; in H2 2.4.240 schaltet das den Hintergrund-Thread und damit jede Kompaktierung ab, wodurch die Dateien im Dauerbetrieb auf Dutzende Gigabyte anwuchsen (gemessen: 37,7 KB Dateiwachstum pro Zeile statt 92,6 Byte Nutzdaten).
2. **Twin-Spiegelung**: Selbst wenn eine der beiden Dateien während eines Schreibvorgangs zerstört wird, bleibt die jeweils andere konsistent.
3. **Auto-Recovery beim Start**: Lässt sich eine der beiden Dateien nicht öffnen (H2-Fehlercode 90030), wird sie in ein `quarantine/`-Unterverzeichnis verschoben und per `Files.copy()` aus der intakten DB rekonstruiert. Anschließend werden beide Connections normal geöffnet – der Betrieb läuft unterbrechungsfrei weiter.
4. **Selbstheilung im Betrieb**: Scheitert ein Lesezugriff auf der Primary an einer Korruption (typisch eine einzelne unlesbare Seite, die das Öffnen nicht verhindert), wird er auf der Shadow wiederholt und eine Marker-Datei `<db>.REBUILD_REQUIRED` gesetzt. Beim nächsten Start vereinigt `DatabaseRebuilder` beide Dateien (nur lesend geöffnet, Tag für Tag, unlesbare Stunden einer Datei deckt die andere ab) in eine frische, mit `SHUTDOWN COMPACT` kompaktierte Datei, verschiebt die alten Dateien nach `quarantine/rebuild_<Zeit>/` und setzt die neue als Primary samt Shadow-Kopie ein. Derselbe Neuaufbau ist manuell über `signalreport.jar rebuild-db` bzw. die Skripte `dbrebuild.bat`/`dbrebuild.sh` auslösbar.

Fehlerszenarien. Die Tabelle 3 dokumentiert das Verhalten bei den verschiedenen Krisenfällen:

Szenario	Verhalten
Crash während Primary-Write	Primary korrupt, Shadow intakt → Beim Neustart wird die Primary aus der Shadow rekonstruiert; maximaler Datenverlust: die Schreibvorgänge der letzten Sekunden auf die Shadow
Crash während Shadow-Write	Shadow korrupt, Primary intakt → Beim Neustart wird die Shadow aus der Primary rekonstruiert; kein Datenverlust
Beide DBs korrupt	Extrem unwahrscheinlich (erfordert Filesystem-Schaden) → Beide Dateien werden in Quarantäne verschoben, Betrieb startet mit leeren Datenbanken neu, Anwendung bleibt funktional
Shadow-Korruption zur Laufzeit	Sehr selten; Shadow wird deaktiviert, Primary läuft weiter, beim nächsten Neustart wird die Shadow automatisch repariert

Tabelle 3: Twin-DB-Verhalten bei Korruptionsfällen

Migration bestehender Installationen. Existiert beim Update auf die neue Version bereits eine alte Single-DB ohne Shadow, wird im ersten Konstruktor-Aufruf die Shadow als exakte Kopie der Primary angelegt. Damit greift die Twin-Redundanz sofort – ohne Datenverlust und ohne manuelle Migrationsschritte.

5.6 Daten-Aufbereitung (Stundenwerte und Aufbewahrung)

Bei einem Messintervall von 10 Sekunden schreibt die Referenzinstallation rund 42 500 Rohzeilen pro Tag. Berichte über Monate aus diesen Zeilen zu rechnen heißt, Millionen Messungen in den Speicher zu laden, und die Datenbank wächst unbegrenzt. Seit Version 2.2.0 verdichtet deshalb ein Hintergrund-Job (`DataPrepScheduler`, Daemon-Thread im Minutentakt) die Rohdaten in zwei Schritten:

- **Stundenwerte**: `RollupService` berechnet je abgeschlossener Stunde, Messtyp und Ziel eine Zeile in `measurement_hourly` (Anzahl Messungen, davon erfolgreich und ausgenommen, Minimum, Durchschnitt, Median, 95. Perzentil, Maximum mit Zeitpunkt, Jitter). Die Wasserstandsmarke `rollup.lastHour` in `rollup_state` merkt die letzte verdichtete Stunde; geschrieben wird per `MERGE`, ein wiederholter Lauf ist daher unschädlich. Jeder Lauf arbeitet auf einem Nebenkanal mit eigenen H2-Verbindungen (`openSideChannel()`) und legt nach jeder Stunde eine kurze Pause ein, damit Messschleife und Web-Oberfläche nie hinter ihm warten (H2 führt je Verbindung nur eine Anweisung gleichzeitig aus; in 2.2.0 teilte sich der Lauf die Verbindungen mit der Messschleife und blockierte sie bei der Erstverdichtung minutenlang). Außerhalb des Zeitfensters werden höchstens zwei Stunden pro Minute verdichtet; im Fenster (Standard 03:00–05:00 oder das Maintenance-Fenster) wird einmal täglich der gesamte Rückstand abgearbeitet, mit Fortschritt im Log und in der Einstellungs-Karte.

- **Aufbewahrung**: Danach löscht `applyRetention()` tageweise – und nur aus bereits verdichteten Stunden – erfolgreiche Rohmessungen, die älter als die Aufbewahrungsfrist sind (Standard 90 Tage, 0 = nie) und deren Vorgänger für denselben Typ und dasselbe Ziel ebenfalls erfolgreich war (Fensterfunktion `LAG(success)`). Fehlgeschlagene Messungen, der jeweils erste Erfolg nach einem Fehlschlag (das Ende eines Ausfalls), ausgenommene Messungen und `MAINTENANCE`-Marker werden nie gelöscht; die Ausfall-Analyse bleibt dadurch exakt. Die Kandidaten eines Tages werden zuerst ermittelt und dann über den Primärschlüssel in Paketen von 1 000 Zeilen gelöscht (kurze Transaktionen; ein einzelnes `DELETE … WHERE id IN (Unterabfrage)` lief in 2.2.0 über die ganze Tabelle und brauchte Minuten pro Tag). Der Fortschritt steht in `retention.doneUntil`.

Die Heatmap liest verdichtete Stunden plus den noch rohen Rest. PDF-Berichte über mehr als 7 Tage (`PdfReportGenerator.RAW_DETAIL_HOURS`) rechnen aus den Stundenwerten: gewichteter Durchschnitt, exaktes Maximum und exakter Paketverlust, 95. Perzentil und Jitter als gewichtete Näherung, Tagesmittel in den Diagrammen, die schlechtesten Stunden statt der schlechtesten Einzelmessungen und `ReliabilityReport.computeFromAggregates()` für die Verfügbarkeit. Der 12-Monats-Bericht ist damit auch bei Jahren an Daten in Sekunden fertig. Der CSV-Export streamt zeilenweise aus dem ResultSet; „alle Daten“ wird als ZIP mit einer CSV-Datei geliefert, zusätzlich gibt es einen Export der Stundenwerte. Bedient wird alles über die Einstellungs-Karte „Daten-Aufbereitung“ (Zeitfenster, Option Maintenance-Fenster, Aufbewahrungstage, Status, „Jetzt ausführen“ mit fünf Minuten Abkühlphase). `DatabaseRebuilder` übernimmt `measurement_hourly` und `rollup_state` in die neu aufgebaute Datenbank.

5.7 Mitgelieferte Listen und eigene Einstellungen

Zwei Listen gehören zum Programm und nicht zur Konfiguration einer Installation: die DNS-Server des Benchmarks und die Dienste der Erreichbarkeitsprüfung. Sie liegen als JSON-Dateien im Programm (`BuiltinCatalog`) und werden nie in `config.json` geschrieben. Jedes Update bringt damit neue, korrigierte und gestrichene Einträge automatisch mit. In `config.json` steht nur, was der Nutzer davon abweichend festlegt:

- **DNS-Benchmark** (`dnsBenchmark`): eigene DNS-Server mit Name, IP-Adresse, Notiz und dem Schalter „im Benchmark“ sowie die abgewählten mitgelieferten Server. Ein voller Lauf prüft die mitgelieferten Server ohne die abgewählten und die eingeschalteten eigenen; „Nur eigene testen“ prüft alle eigenen. Angenommen werden nur IP-Adressen, keine Hostnamen. Die Prüfung löst nie per DNS auf: IPv4 muss einem strengen Muster entsprechen, IPv6 wird in eckigen Klammern geparst, und `InetAddress.getAllByName` behandelt beides laut JDK-Quelltext nur als Literal.
- **Dienst-Erreichbarkeit** (`serviceReachability.serviceOverrides`): nur die Schalter, die vom mitgelieferten Standard abweichen. Neue Dienste eines Updates erscheinen mit ihrem Standard-Schalter.

Ältere `config.json`-Dateien mit den kompletten Listen werden beim ersten Laden überführt: DNS-Einträge, die nicht mitgeliefert sind, werden eigene Server; Dienst-Schalter, die vom Standard abweichen, werden gemerkte Abweichungen. Es geht nichts verloren.

5.8 Internationalisierung (i18n)

Die gesamte Präsentationsschicht – Web-Oberfläche, Login- und Setup-Seite, PDF-Berichte, CSV-Spaltenköpfe und benutzerseitige API-Meldungen – ist mehrsprachig. Aktuell werden neun Sprachen mitgeliefert: Deutsch (Referenz), Englisch, Französisch, Italienisch, Spanisch, Portugiesisch, Türkisch, Polnisch und Ukrainisch.

Sprachdateien. Jede Sprache ist eine flache JSON-Datei (UTF-8) mit Punkt-Schlüsseln unter `resources/lang/`, z. B. `nnav.settings: Ëinstellungen"`. Die zentrale Klasse `I18n` lädt die in der Konfiguration hinterlegte Sprache und stellt drei Mechanismen bereit:

1. **Platzhalter-Auflösung**: Die HTML-Renderer enthalten statt deutscher Literale Platzhalter der Form `{{nav.settings}}`. `I18n.resolve()` ersetzt sie beim Rendern per regulärem Ausdruck.
2. **JS-Injektion**: Texte, die das Frontend-JavaScript zur Laufzeit erzeugt (Status-Badges, Chart-Beschriftungen, Fehlermeldungen), werden als `const I18N = {...}`-Objekt in die Seite eingebettet. Datums- und Zahlenformate folgen über die mitgereichte Locale der gewählten Sprache.
3. **Direkte Aufrufe**: PDF-Generator und WebServer verwenden `I18n.get(key)` sowie locale-bewusste Formatierung (`String.format(locale, ...)`) – im Deutschen erscheint `23,5 ms`, im Englischen `23.5 ms`.

Fallback-Kette. Fehlt ein Schlüssel in der gewählten Sprache, wird auf Deutsch (Referenzsprache) zurückgegriffen, danach auf den Schlüsselnamen selbst – die Oberfläche zeigt also nie ein Loch. Ein eigener Testfall (`I18nTest`) erzwingt zusätzlich, dass alle mitgelieferten Sprachdateien exakt dieselben Schlüssel wie die Referenz enthalten und dass jeder im Quellcode verwendete Platzhalter existiert.

Erweiterbarkeit ohne Neukompilieren. Neben den im JAR gebündelten Sprachen liest `I18n` einen externen Ordner `./lang` neben der JAR-Datei ein. Eine dort abgelegte Datei (z. B. `ca.json` für Katalanisch) erscheint automatisch im Sprach-Dropdown; gleichnamige Dateien überschreiben gebündelte Sprachen, womit auch Übersetzungskorrekturen ohne Neubau möglich sind. Der Anzeigename jeder Sprache stammt aus ihrem eigenen Schlüssel `language.name` und erscheint im Dropdown in der jeweiligen Eigenbezeichnung (Deutsch, English, Türkçe, Ukrainisch in kyrillischer Schrift usw.).

Sprachwahl. Die Sprache wird global pro Installation in `config.json` gespeichert. Sie ist an zwei Stellen umstellbar: per Dropdown im Setup-Wizard (noch vor der Passwortvergabe, via `/setup?lang=xx`) und im Einstellungen-Tab (sofortige Übernahme über `POST /api/config/language` mit anschließendem Seiten-Reload). Neuinstallationen starten in der Systemsprache des Rechners, sofern unterstützt – andernfalls auf Englisch. Bestehende Konfigurationen ohne `language`-Feld bleiben unverändert auf Deutsch.

PDF und Sonderzeichen. Die eingebauten PDF-Schriften (Helvetica) beherrschen nur Westeuropa-Zeichen; Türkisch, Polnisch und Ukrainisch wären damit im Bericht nicht darstellbar. Deshalb wird die freie Schrift **DejaVu Sans** (Regular, Bold, Oblique) als

Ressource mitgeliefert und mit Unicode-Encoding (`IDENTITY_H`) in jedes PDF eingebettet. Dieselben TTF-Dateien dienen JFreeChart als AWT-Schrift, sodass auch Chart-Titel und Achsenbeschriftungen in allen Sprachen korrekt gerendert werden. Der JAR-Zuwachs durch Schriften und neun Sprachdateien beträgt rund 1,5 MB.

5.9 Datenfluss bei der Messung

Der Messablauf (Abbildung 4) zeigt den kontinuierlichen Zyklus:

1. Konfiguration laden und Maintenance-Fenster prüfen
2. Externe IP ermitteln und IP-Änderungen protokollieren
3. Drei Messungen ausführen (Ping, DNS, HTTP)
4. Ergebnisse in der Datenbank speichern
5. Konfigurierbares Intervall abwarten

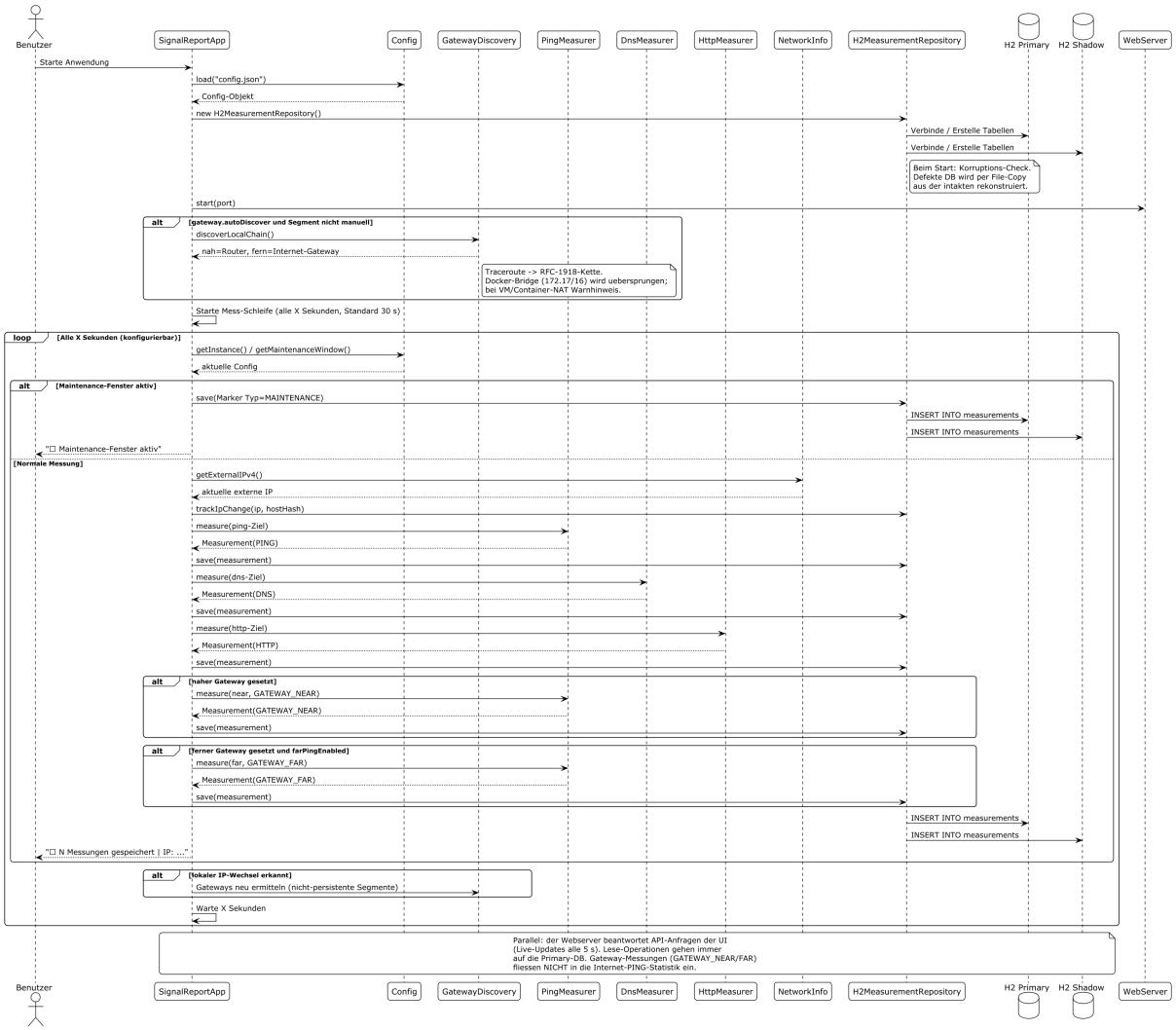


Abbildung 4: Sequenzdiagramm: Messablauf

5.10 PDF-Export-Prozess

Der PDF-Bericht (Abbildung 5) durchläuft mehrere Schritte: Statistik-Berechnung, Chart-Generierung mit JFreeChart, Zieländerungs-Erkennung, Ausfall-Analyse und abschliessende PDF-Zusammenstellung mit OpenPDF.

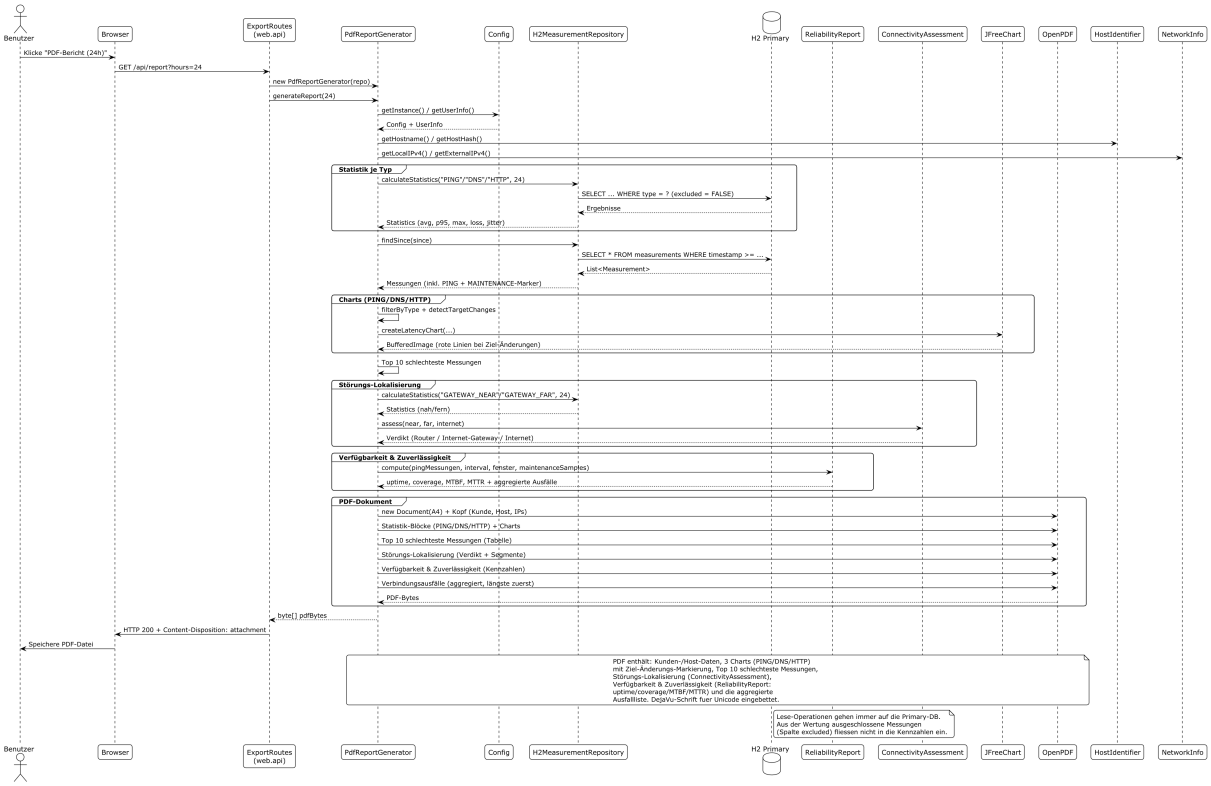


Abbildung 5: Sequenzdiagramm: PDF-Export

5.11 Use-Cases

Das Use-Case-Diagramm (Abbildung 6) zeigt alle 16 Anwendungsfälle, aufgeteilt auf die drei Akteure Benutzer, Administrator und System. Die vollständigen User Stories zu jedem Use Case finden sich in Abschnitt 3.

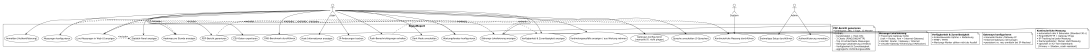


Abbildung 6: Use-Case-Diagramm von SignalReport

5.12 Authentifizierungs-Zustände

Das State-Diagramm (Abbildung 7) modelliert den Lebenszyklus der Authentifizierung: vom initialen Setup über den öffentlichen Zugriff bis zum geschützten Modus mit Admin- und User-Rollen.

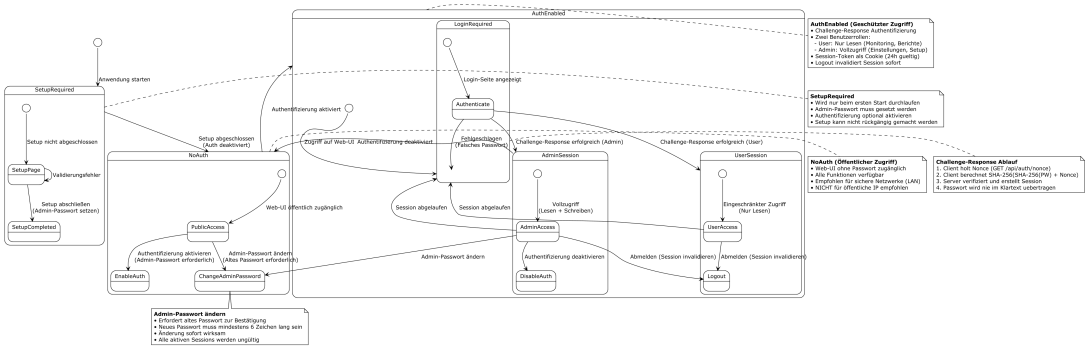


Abbildung 7: State-Diagramm: Authentifizierung

6 Implementierung

6.1 Mess-Engine und Strategy-Pattern

Die Mess-Engine basiert auf dem `Measurer`-Interface, das alle Messtypen vereinheitlicht (Strategy-Pattern). Jede Implementierung hat eine einzige Methode `measure(String target)`, die ein `Measurement`-Objekt zurückgibt:

Listing 1: Measurer-Interface

```
1 public interface Measurer {
2     Measurement measure(String target) throws Exception;
3 }
```

Die drei Implementierungen:

PingMeasurer Nutzt `InetAddress.isReachable(3000)` für ICMP-Echo-Tests. Misst die Round-Trip-Time in Nanosekunden und konvertiert zu Millisekunden.

DnsMeasurer Nutzt `InetAddress.getByName(hostname)` und misst die Zeit der DNS-Auflösung.

HttpMeasurer Nutzt `java.net.http.HttpClient` mit konfigurierbarem Timeout (5 Sekunden). Misst die Gesamtzeit des HTTP-GET-Requests. Jede empfangene Antwort zählt als Erfolg, auch 4xx und 5xx: Sie belegt, dass Verbindung, TLS und HTTP bis zum Server und zurück funktionieren; bewertet wird die Leitung, nicht die Website. Fehlschlag sind nur Timeout und Verbindungsfehler; der Grund wird je Ursache höchstens alle zehn Minuten ins Log geschrieben (seit 2.2.1; bis 2.2.0 galten nur 2xx/3xx und der Grund eines Fehlschlags verschwand stillschweigend).

Listing 2: PingMeasurer-Implementierung

```
1 public class PingMeasurer implements Measurer {
2     public Measurement measure(String target) throws Exception {
3         long start = System.nanoTime();
4         boolean reachable = InetAddress.getByName(target).isReachable
              (3000);
5         long end = System.nanoTime();
6         double latency = (end - start) / 1_000_000.0;
```

```
7           return new Measurement(target, latency, reachable, "PING");
8       }
9   }
```

6.2 IP-Tracking und Host-Identifikation

Jede Messung speichert zusätzlich Netzwerk-Informationen:

- Lokale IPv4- und IPv6-Adressen
- Externe IPv4- und IPv6-Adressen (über ipify.org, mit 120-Sekunden-Cache)
- Eindeutiger Host-Hash (MD5 aus Hostname + OS + MAC-Adresse)

Die IP-Tracking-Logik erkennt automatisch Änderungen der externen IP und protokolliert sie in der `ip_changes`-Tabelle:

Listing 3: IP-Aenderungserkennung

```
1  public void trackIpChange(String currentIp, String hostHash)
2          throws SQLException {
3      String lastIp = getLastKnownIp(hostHash);
4      if (lastIp == null) {
5          recordIpChange(null, currentIp, "INITIAL", hostHash);
6      } else if (!lastIp.equals(currentIp)) {
7          recordIpChange(lastIp, currentIp, "CHANGE", hostHash);
8      }
9  }
```

Hinweis zu IPv6 (DS-Lite): Bei Providern mit DS-Lite (z. B. Magenta-Austria) wird die externe IPv6-Adresse als "unknown" angezeigt, da diese Provider gemäß RFC 8415 keine stabile IPv6-Präfixdelegation anbieten.

6.3 Statistische Auswertung

Die statistische Auswertung berechnet vier Kennzahlen für jeden Messtyp:

- **Durchschnittliche Latenz**: $\mu = \frac{1}{n} \sum_{i=1}^{n} x_i$
- **95. Perzentil**: Wert, unter dem 95 % der Messungen liegen – ein robusterer Indikator als der Durchschnitt, da er Ausreißer toleriert
- **Paketverlust**: $\frac{\text{fehlgeschlagene Messungen}}{\text{gesamte Messungen}} \times 100$
- **Jitter**: Durchschnitt der absoluten Differenzen zwischen aufeinanderfolgenden Messungen – zeigt die Stabilität der Verbindung

6.4 Web-Interface und Rendering-Architektur

Das Web-Interface ist mit Javalin 5.6.3 implementiert. Um die Trennung von Verantwortlichkeiten zu wahren, wurde das HTML-Rendering in eigenständige Klassen ausgelagert:

WebServer (ca. 140 Zeilen) – Orchestrator: richtet Javalin ein, installiert zwei `before`-Filter (Setup- und Auth-Gating) und delegiert die Routen an die neun Registrar-Klassen im Paket `web.api` (`PageRoutes`, `MeasurementRoutes`, `ReliabilityRoutes`, …), die je eine statische `register(app, …)`-Methode bereitstellen.

web.view-Renderer – `HtmlPageRenderer` (ca. 490 Zeilen) generiert die Hauptseite, `SetupPageRender` den Setup-Assistenten und `LoginPageRenderer` die Login-Seite. Das umfangreiche CSS und JavaScript wurde aus dem `HtmlPageRenderer` in statische Ressourcen (`app.css`, `app.js`) ausgelagert; im Renderer verbleiben die HTML-Struktur mit `{{key}}`-Platzhaltern und ein kleines Inline-Skript mit den server-injizierten Globals (`I18N`, `LOCALE`, `GW_LABELS`).

SessionManager (ca. 140 Zeilen) – Verwaltet Nonces (Single-Use, 60 s TTL), Sessions (24 h Timeout) und verifiziert Challenge-Responses.

6.4.1 Dark Mode

Das Theming basiert auf CSS Custom Properties (Variablen), die in zwei Blöcken definiert sind:

Listing 4: CSS-Variablen für Light/Dark Mode

```
:root {
    --bg-body: #f8f9fa;
    --bg-card: #ffffff;
    --color-text: #212529;
    /* ... weitere Variablen ... */
}
body.dark-mode {
    --bg-body: #1a1a2e;
    --bg-card: #16213e;
    --color-text: #e0e0e0;
    /* ... weitere Variablen ... */
}
```

Um ein *Flackern* beim Laden zu vermeiden (Flash of Wrong Theme), wird die CSS-Klasse server-seitig injiziert:

Listing 5: Server-seitige Theme-Injection

```
boolean darkMode = Config.getInstance().getTheme().isDarkMode();
String bodyClass = darkMode ? " class=\"dark-mode\"" : "";
// ... im HTML-Template:
return """<body""" + bodyClass + """>...</body>""";
```

Chart.js-Diagramme reagieren nicht auf CSS-Variablen, daher werden deren Farben (Achsenbeschriftung, Gitterlinien) explizit per JavaScript beim Theme-Wechsel aktualisiert.

6.4.2 DNS-Benchmark

Der DNS-Benchmark vergleicht konfigurierbare DNS-Server weltweit (Google, Cloudflare, Quad9, etc.) per paralleler Abfrage. Dabei kommen **Virtual Threads** (Java 21) zum

Einsatz: Über `Executors.newVirtualThreadPerTaskExecutor()` wird für jeden DNS-Server ein eigener virtueller Thread erstellt, der die Abfrage unabhängig durchführt. Dies ermöglicht echte Parallelität ohne den Overhead klassischer Plattform-Threads. Ein 30-Sekunden-Cooldown verhindert Missbrauch. Die Ergebnisse werden nach Latenz sortiert und mit regionaler Farbcodierung dargestellt.

6.4.3 Challenge-Response Authentifizierung

Anstelle von HTTP Basic Auth, bei dem das Passwort im Klartext übertragen wird, verwendet SignalReport ein Challenge-Response-Verfahren auf Basis von SHA-256. Der Ablauf:

1. Client fordert eine Einmal-Nonce an (`GET /api/auth/nonce`)
2. Client berechnet: `SHA-256(SHA-256(Passwort) + Nonce)`
3. Server berechnet: `SHA-256(gespeicherterHash + Nonce)`
4. Bei Übereinstimmung wird ein Session-Token erstellt (24 h gültig)
5. Alle weiteren Requests werden per Session-Cookie (`SR_SESSION`) authentifiziert

Nonces sind einmalig verwendbar (Single-Use) und verfallen nach 60 Sekunden. Die kryptografischen Operationen im Browser werden über die Web Crypto API (`crypto.subtle.digest`) durchgeführt – damit ist kein Klartext-Passwort in der Netzwerkkommunikation enthalten.

Die Session-Verwaltung erfolgt über die Klasse `SessionManager`, die Nonces und Sessions in `ConcurrentHashMap`-Instanzen verwaltet.

6.5 PDF-Export

Der PDF-Export mit OpenPDF und JFreeChart generiert professionelle Berichte mit:

- Kundendaten (Name, Provider, Kundennummer)
- Host-Informationen (Hostname, Hash, IPs)
- 3 Charts (PING/DNS/HTTP) mit roten Linien bei Ziel-Änderungen
- Chronologische Ziel-Liste unter jedem Chart
- Top 10 schlechteste Messungen (sortiert nach Latenz)
- Verbindungsausfälle mit Gesamtanzahl
- Top 10 längste Verbindungsausfälle

Berichte können für drei Zeiträume generiert werden: 24 Stunden, 7 Tage und 12 Monate. Bis 7 Tage (`RAW_DETAIL_HOURS` = 168) rechnet der Bericht aus den Rohmessungen, darüber aus den Stundenwerten der Daten-Aufbereitung (Abschnitt 5.6): Tagesmittel in den Diagrammen, gewichtete Statistik, die schlechtesten Stunden statt der schlechtesten Einzelmessungen und die Ausfall-Analyse aus den aufbewahrten Fehlschlägen. Im Smoke-Test entstand ein 12-Monats-Bericht über 120 Tage Testdaten (51 840 Rohzeilen, 2 880 Stundenwerte) in rund 1,1 Sekunden. Die Berichte sind direkt als Anlage für Provider-Beschwerden verwendbar.

6.6 Konfigurationsmanagement

Die `Config`-Klasse ist als Singleton implementiert; jeder Konfigurationsbereich liegt seit dem Refactoring in einer eigenen Datei im Paket `config` (statt als verschachtelte statische Klasse):

Listing 6: Config-Struktur (Auszug)

```
public class Config {
    private MeasurementConfig measurement;
    private GatewayConfig gateway;       // Stoerungs-Lokalisierung
    private ServiceReachabilityConfig serviceReachability;  // Dienst
       -Erreichbarkeit (V2)
    private MaintenanceWindow maintenanceWindow;
    private DataPrepConfig dataPrep;     // Daten-Aufbereitung
       (2.2.0)
    private UserInfo userInfo;
    private SetupConfig setup;
    private AuthConfig auth;
    private ThemeConfig theme;  // Dark Mode
    private DnsBenchmarkConfig dnsBenchmark;  // nur eigener Teil
       (2.3.0)

    public static String hashPassword(String password) {
        // SHA-256 Hashing -- zentral, nicht dupliziert
    }
}
```

Die Persistenz erfolgt über Jackson JSON-Serialisierung in `config.json`. Änderungen über die Web-UI werden sofort gespeichert.

Mitgelieferte Listen. Die DNS-Server des Benchmarks und die Dienste der Erreichbarkeitsprüfung liegen seit Version 2.3.0 als JSON-Dateien im Programm (`/catalog/dns-servers.json`, `/catalog/services.json`, gelesen von `BuiltinCatalog`) und nicht mehr in `config.json`. Dort steht nur der eigene Teil: eigene DNS-Server, abgewählte mitgelieferte Server und die Dienst-Schalter, die vom Standard abweichen (Abschnitt 5.7). Bis 2.2.1 wurden beide Listen beim ersten Start komplett in `config.json` geschrieben; Ergänzungen im Code erreichten bestehende Installationen deshalb nie. Beim ersten Laden einer solchen Datei überführt `Config.migrateLegacyLists()` die alten Listen und schreibt `config.json` im neuen Format.

6.7 Datenbank-Schema

H2 wird im FILE-Modus mit Twin-Architektur betrieben: zwei parallele Datenbanken `signalreport.mv.db` (Primary) und `signalreport-shadow.mv.db` (Shadow). Beide nutzen das identische Schema. Die JDBC-URL belässt `WRITE_DELAY` auf dem H2-Standard (500 ms); alle Messungen einer Runde werden in einer Transaktion geschrieben (`saveAll`, siehe Abschnitt 5.5).

Sechs Tabellen bilden das Schema. `findSince(Instant)` liefert Zeiträume bis 7 Tage für Berichte, `forEachMeasurement(…)` streamt beliebige Zeiträume zeilenweise für den CSV-Export (ohne alle Zeilen im Speicher zu halten), `findLastN(int)` dient der Live-Anzeige im Web-Interface, und die Stundenwerte werden über `findHourlyRollups` bzw.

`calculateStatisticsFromRollups` gelesen. Lese-Operationen gehen auf die Primary-Datenbank und weichen bei einem Korruptionsfehler auf die Shadow aus (Abschnitt 5.5):

Tabelle	Primärschlüssel	Inhalt
`measurements`	id (BIGINT)	Zeitstempel, Ziel, Latenz, Erfolg, Typ (PING/DNS/HTTP/GATEWAY_NEAR/GATE IPv4/v6, Host-Hash, `excluded` (aus der Wertung genommen)
`hosts`	host_hash (VARCHAR)	Hostname, Betriebssystem, Erstregistrierung, letzte Aktivität
`ip_changes`	id (BIGINT)	Zeitstempel, alte/neue IP, Änderungstyp, Host-Hash
`service_checks`	id (BIGINT)	Zeitstempel, Dienst-ID, Verdikt (REACHABLE / DNS_BLOCKED / SNI_BLOCKED / …), Methode, HTTP-Status, aufgelöste IP, Latenz – für die Dienst-Erreichbarkeit (Version 2.0)
`measurement_hourly`	hour_start, type, target	Stundenwert: Anzahl Messungen, davon erfolgreich und ausgenommen, Min/Durchschnitt/Median/P95/Max (ms), Zeitpunkt des Maximums, Jitter, Host-Hash – Grundlage für Heatmap, lange Berichte und Stunden-CSV (Version 2.2.0)
`rollup_state`	state_key (VARCHAR)	Zustand der Daten-Aufbereitung: `rollup.lastHour` (letzte verdichtete Stunde), `retention.doneUntil` (Aufbewahrung angewendet bis)

Tabelle 4: H2-Datenbank-Schema

Für performante Abfragen bei wachsender Datenmenge werden beim Start automatisch Indizes auf den Spalten `timestamp`, `type`, `host_hash` sowie ein kombinierter Index `(type, timestamp)` sowie ein Index `service_checks(service_id, timestamp)` angelegt.

7 Test

7.1 Teststrategie

Die Teststrategie umfasste:

- **Unit-Tests**: Für die Kernkomponenten (z. B. Statistik-Berechnung, Config-Handling, Measurer-Interface)

- **Integrationstests**: Für die Datenbankanbindung und API-Endpunkte
- **E2E-Tests**: Für das Web-Interface und PDF-Export
- **Manuelle Tests**: Für UI-Interaktionen und Benutzerfreundlichkeit

7.2 JUnit-Testabdeckung

SignalReport wurde mit JUnit 5 umfassend getestet. Die 22 Testklassen mit laufenden Tests – in Paketen, die die Quellstruktur spiegeln (`config`, `measurement`, `network`, `storage`, `report`, `web`, `i18n`) – mit insgesamt 168 Tests (zuzüglich zweier netz- bzw. PDF-gebundener Smoke-Klassen, die per Umgebungsvariable aktivierbar sind) decken folgende Bereiche ab:

- **MeasurementTest** (5 Tests): Grundlegende Objekt-Erstellung und Validierung
- **HostIdentifierTest** (4 Tests): Stabile Host-Identifikation über Neustarts
- **StatisticsTest** (8 Tests): Mathematisch korrekte Berechnung von 95. Perzentil, Jitter und Paketverlust mit realen DB-Abfragen
- **ConfigTest** (18 Tests): Laden/Speichern von Konfiguration, hashPassword, Auth-Verifikation, Theme, Save+Reload, Standardwerte und Aktualisierung der Daten-Aufbereitung
- **H2MeasurementRepositoryTest** (13 Tests): Vollstaendige Datenbank-Integration mit IP-Tracking, Host-Registrierung, Messrunde als eine Transaktion (`saveAll`), Nebenkanal mit eigenen Verbindungen (sieht und schreibt dieselben Daten, schließt unabhängig)
- **HttpMeasurerTest** (3 Tests): HTTP-Messung gegen einen echten lokalen Server – 200, 302, 403, 404 und 503 zählen als erreichbar, verweigerte Verbindung als Fehlschlag, Fehlergründe benannt
- **MeasurerInterfaceTest** (6 Tests): Interface-Implementierung, Polymorphie, reale Messungen (Ping/DNS/HTTP)
- **MaintenanceWindowTest** (7 Tests): Deaktiviert, ganztägig, aktuelle Zeit, Mitternachtsübergang, Standardwerte
- **SessionManagerTest** (19 Tests): Nonce-Generierung und -Validierung (Single-Use, Ablauf), Session-Erstellung und -Timeout (24 h), Rollenunterscheidung (Admin/User), Challenge-Response-Verifikation, Replay-Schutz
- **I18nTest** (10 Tests): Schlüssel-Parität aller neun Sprachdateien gegen die Referenz (de.json), Existenz aller im Quellcode verwendeten Platzhalter (inkl. `app.js`), Fallback-Verhalten, Sprach-Erkennung
- **GatewayDiscoveryTest** (15 Tests): Traceroute-Parsing, RFC-1918-Klassifizierung, Gateway-Kette (nah/fern), Erkennung virtueller Gateways (Docker-Bridge, VM-NAT) und Überspringen der Docker-Bridge

- **ConnectivityAssessmentTest** (8 Tests): Schuld-Verdikt (Router / Internet-Gateway / Internet) auf Basis des Paketverlusts je Segment
- **ReliabilityReportTest** (13 Tests): lückenbewusste Verfügbarkeit/Abdeckung/MTBF/MTTR, Aggregation aufeinanderfolgender Fehlmessungen zu einem Ausfall, Ausschluss via `excluded`-Spalte, Auswertung aus Stundenwerten (`computeFromAggregates`) deckungsgleich mit der Rohdaten-Rechnung
- **ServiceReachabilityConfigTest** (11 Tests): Defaults (deaktiviert, 6-Stunden-Takt), Dienstliste aus der mitgelieferten Liste, Intervall-Grenzen, Pro-Dienst-Schalter als Abweichung vom Standard, Übernahme alter Listen, eigene Dienste, JSON-Round-Trip ohne komplette Liste
- **BuiltinCatalogTest** (4 Tests): mitgelieferte DNS- und Dienstliste vollständig lesbar, nur gültige IP-Adressen ohne Dubletten, eindeutige Dienst-ids, Herausgabe als Kopie
- **DnsBenchmarkConfigTest** (8 Tests): Adressprüfung ohne DNS-Auflösung (IPv4, IPv6, Hostnamen, Zonen, Sonderadressen), Zusammenstellung für vollen Lauf und „Nur eigene testen“, Prüfung neuer Einträge mit Altbestand, Übernahme alter `config.json`-Dateien
- **ServiceReachabilityAssessmentTest** (15 Tests): Verdikt-Klassifizierung (erreichbar / Dienst-Störung / DNS-, TCP-, SNI-Sperre / Sperrseite) aus synthetischen Beobachtungen
- **ServiceReachabilityReportTest** (4 Tests): Verdichtung der Prüfungen zu Episoden (die Sperr-/Störungs-Timeline)
- **ServiceReachabilitySchedulerTest** (5 Tests): Leitungs-Gate-Entscheidung (nur erfolgreiche PING-/HTTP-Messungen zählen als Internet-Beleg)
- **ServiceCheckRepositoryTest** (3 Tests): `service_checks`-Persistenz, Zeitraum-Abfrage, neueste Prüfung je Dienst
- **ServiceReachabilityRoutesTest** (2 Tests): „Jetzt prüfen“-Endpoint inkl. Abkühlphase (gegen echten Javalin-Server)
- **SystemRoutesTest** (2 Tests): lokaler Stopp-Endpoint (gegen echten Javalin-Server) und Loopback-Erkennung (IPv4/IPv6-Literale, keine Hostnamen-Auflösung)
- **StopCommandTest** (3 Tests): Stopp-Kommando `signalreport.jar stop` beendet eine laufende Instanz über den Endpoint; Randfälle „keine Instanz“ und „keine Konfiguration“
- **DatabaseRebuilderTest** (4 Tests): Neuaufbau mit echten H2-Dateien – Vereinigung von Primary und Shadow inkl. `excluded`-Flag, Übernahme der Stundenwerte und der Wasserstandsmarke, Stunden-Fallback bei simuliert unlesbarem Tag, Dateitausch mit Quarantäne und Shadow-Kopie, Marker-gesteuerter Neuaufbau beim Repository-Start, Abbruch ohne lesbare Quelle
- **RebuildCommandTest** (1 Test): Pfad-Fallback des Kommandozeilen-Neuaufbaus ohne Konfiguration

- **DataPrepConfigTest** (10 Tests): Zeitfenster (einfach, über Mitternacht, Start = Ende), Umschaltung auf das Maintenance-Fenster mit Rückfall, Begrenzung der Aufbewahrungsfrist (7–3650 Tage, 0 = nie), Begrenzung von Stunde/Minute, JSON-Round-Trip
- **DataPrepSchedulerTest** (5 Tests): Entscheidung „voller Lauf einmal pro Tag im Fenster“, Obergrenze außerhalb des Fensters, voller Lauf gegen echte H2-Datei inkl. Status und Fortschritt, manueller Start mit Abkühlphase, sofortige Abweisung eines zweiten Starts während ein Lauf aktiv ist (keine blockierende Sperre)
- **RollupServiceTest** (12 Tests): Stundenstatistik ohne DB (Min/Ø/Median/P95/Max/Jitter; Fehlschläge und ausgenommene Messungen), Verdichtung nur abgeschlossener Stunden, Wasserstandsmarke und Stundenlimit, ausstehende Stunden und Fortschrittsmeldung, Wiederholbarkeit (`MERGE`), Heatmap aus Stundenwerten plus Rohrest, Aufbewahrungsregel (Funde, Ausfall-Ende, ausgenommene Messungen und Wartungs-Marker bleiben; nur verdichtete Stunden; Tageslimit pro Lauf)
- **ExportRoutesTest** (7 Tests): gestreamter CSV-Export, Typfilter, Parameterfehler, „alle Daten“ als ZIP mit genau einer CSV-Datei, Stundenwerte-CSV (gegen echten Javalin-Server), CSV-Escaping und Zeilenformat
- **DnsRoutesTest** (5 Tests): DNS-Endpunkte gegen echten Javalin-Server mit eigener Konfiguration – Katalog, Speichern eigener Server und abgewählter mitgelieferter, lesbare Ablehnung eines Hostnamens, fehlerhafter Anfrage-Körper, „Nur eigene testen“ ohne eigene Server (ohne Netzverkehr)

Alle Tests laufen ohne externe Abhängigkeiten (H2-Datenbankdateien unter `data/test-*` für Integrationstests) und sind in wenigen Sekunden durchführbar.

7.3 Beispiel-Test für Statistik-Berechnung

Listing 7: Test fuer 95. Perzentil

```
@Test
void testP95CalculationWithSortedLatencies() {
    // Arrange: 20 Messungen (95. Perzentil = 19. Wert)
    List<Double> latencies = new ArrayList<>();
    for (int i = 1; i <= 20; i++) {
        latencies.add((double) i * 10); // 10, 20, 30, ..., 200
    }

    // Act: 95. Perzentil = Index 18 (0-basiert) = 190.0
    int p95Index = (int) Math.ceil(latencies.size() * 0.95) - 1;
    double p95Value = latencies.get(p95Index);

    // Assert
    assertEquals(18, p95Index, "Index fuer 95. Perzentil muss 18 sein
       ");
    assertEquals(190.0, p95Value, 0.01, "95. Perzentil-Wert muss
       190.0 sein");
}
```

7.4 Testergebnisse

Testkategorie	Anzahl	Erfolgsquote
Unit-Tests (Measurement, HostIdentifier, Config, Measurer, MaintenanceWindow, DataPrepConfig, BuiltinCatalog, DnsBenchmarkConfig, SessionManager, I18n, GatewayDiscovery, ConnectivityAssessment, ReliabilityReport, ServiceReachabilityConfig, ServiceReachabilityAssessment, ServiceReachabilityReport, ServiceReachabilityScheduler, RebuildCommand, DataPrepSchedulerEntscheidung)	165	100%
Integrationstests (Statistics, H2MeasurementRepository, ServiceCheckRepository, DatabaseRebuilder, RollupService, DataPrepScheduler, HttpMeasurer, ServiceReachabilityRoutes, SystemRoutes, ExportRoutes, DnsRoutes, StopCommand)	65	100%
E2E-Tests (manuell: Web-UI, PDF-Export, CSV-Export)	5	100%
Smoke-Tests (opt-in: Netz-Probe, PDF-Erzeugung 24 h und 12 Monate)	4	–
Gesamt	**230 + 5**	**100%**

Tabelle 5: Testergebnisse

7.5 Testumgebung

- **Betriebssysteme**: Windows 10 64-Bit, Windows 11, Windows 11 on ARM, Raspberry OS, Debian 13.4, Ubuntu 25.10, macOS Tahoe 26, Synology DSM 7.3.2, Docker Desktop 4.67.0
- **Java-Version**: 21
- **Browser**: Chrome, Chromium, Vivaldi, Firefox, Edge, Opera, Safari
- **Hardware**: Desktop-PC (Intel Core i7 14. Gen.), Desktop-PC (AMD Ryzen 5 7040 Series), Notebook (Intel Core Ultra i7 100 Serie), Raspberry Pi 4, Raspberry Pi 5, Synology NAS DS420j, MacBook Air M2

7.6 Test-Philosophie und -Qualität

SignalReport verwendet eine gezielte Test-Strategie:

- **Kernfunktionalität priorisiert**: Tests fokussieren auf kritische Geschäftslogik (Statistik-Berechnung, IP-Tracking, Measurer-Polymorphie), nicht auf Debug-Hilfen wie `toString()`
- **Realistische Szenarien**: Tests simulieren echte Nutzung (z. B. IP-Wechsel mit korrekter Zeitreihen-Sortierung, Wartungsfenster über Mitternacht)

- **Wartbarkeit**: Tests vermeiden fragile Assertions (z. B. exakte String-Formate) zugunsten stabiler, aussagekräftiger Prüfungen
- **100% Bestehensquote**: Alle 168 automatisierten Tests laufen reproduzierbar grün – nachweisbare Qualität für die Prüfung

8 Fazit und Ausblick

8.1 Zusammenfassung

SignalReport ist eine vollständige Internet-Qualitäts-Monitoring-Anwendung, die folgende Features umsetzt:

- Kontinuierliche Messung von Ping, DNS und HTTP-Latenz
- Web-basierte Visualisierung mit Live-Charts und Heatmap
- Statistische Auswertung (95. Perzentil, Jitter, Paketverlust)
- PDF-Export für Berichte (inkl. 12-Monats-Berichte für Provider)
- CSV-Export für Excel-Analysen
- IP-Tracking für dynamische IP-Adressen
- Konfigurierbare Messziele und Maintenance-Fenster
- Challenge-Response-Authentifizierung (SHA-256) für sicheren Zugriff
- Setup-Wizard für benutzerfreundliche Ersteinrichtung
- Crash-resistente Datenpersistenz durch Twin-Datenbank-Architektur
- Mehrsprachige Oberfläche, PDF- und CSV-Ausgabe (neun Sprachen)

8.2 Gelerntes

Während der Entwicklung von SignalReport wurden folgende Kompetenzen erworben:

- **Java 21**: Nutzung moderner Features (Virtual Threads (z. B. DNS-Benchmark), Pattern Matching, Text Blocks)
- **Web-Entwicklung**: Javalin-Framework, REST-API, Chart.js
- **Datenbank-Programmierung**: H2-Datenbank, SQL, JDBC
- **PDF-Generierung**: OpenPDF und JFreeChart für professionelle Berichte
- **Testing**: JUnit 5, Mockito, Test-Driven Development
- **Software-Architektur**: UML-Diagramme, Design-Patterns, Clean Code
- **Projektmanagement**: Versionskontrolle mit Git, Dokumentation mit LaTeX

8.3 Herausforderungen und Lösungen

- **Challenge**: IP-Tracking bei dynamischen IPs
  Lösung: Separate Tabelle `ip_changes` mit Zeitstempeln und Change-Type
- **Challenge**: Maintenance-Fenster mit Über-Mitternacht-Unterstützung
  Lösung: Logik zur Erkennung von Zeitfenstern, die über 00:00 Uhr gehen
- **Challenge**: PDF-Generierung mit mehreren Charts und Tabellen
  Lösung: OpenPDF statt PDFBox für bessere Schriftarten-Unterstützung
- **Challenge**: Setup-Wizard ohne Kommandozeilen-Abhängigkeit
  Lösung: Web-basierte Setup-Seite mit Middleware-Integration
- **Challenge**: Passwort-Sicherheit ohne HTTPS
  Lösung: Challenge-Response-Authentifizierung mit SHA-256 und Einmal-Nonces – Passwort wird nie im Klartext übertragen
- **Challenge (nach Diplomprüfung)**: Datenbank-Korruption durch Windows-Update-Neustart mitten im Schreibvorgang – Recovery-Tool fand keinen einzigen konsistenten Chunk
  Lösung: Twin-Datenbank-Architektur mit synchroner Spiegelung und automatischer Rekonstruktion aus der intakten Datei beim Neustart (siehe Abschnitt 5.5)

8.4 Mögliche Erweiterungen

- **HTTPS-Unterstützung**: Verschlüsselte Kommunikation über TLS-Zertifikate
- **Automatischer täglicher PDF-Export**: Cron-Job für regelmäßige Berichte
- **Prometheus Exporter**: Integration in bestehende Monitoring-Tools (Grafana)
- **Multi-Host-Vergleich**: Vergleich von Messungen über mehrere Instanzen
- **Mobile-App**: Native iOS/Android-App für unterwegs

8.5 Fazit

SignalReport demonstriert, dass eine professionelle Monitoring-Lösung nicht teuer oder komplex sein muss. Mit modernen Java-Technologien und einer durchdachten Architektur entstand eine Anwendung, die:

- **Praktisch nutzbar** ist für echte Provider-Beschwerden
- **Technisch sauber** implementiert ist mit Tests und Dokumentation
- **Erweiterbar** bleibt für zukünftige Anforderungen
- **Lehrreich** war für die persönliche Weiterentwicklung

8.6 Weiterentwicklung nach der Diplomprüfung

Mit der erfolgreichen Präsentation im April 2026 endete der Diplomlehrgang – nicht aber das Projekt. SignalReport wird seither als persönliches Open-Source-Vorhaben aktiv weiterentwickelt. Bereits umgesetzte Verbesserungen über den Diplomstand hinaus:

- **Twin-Datenbank-Architektur**: Synchrone Spiegelung und Auto-Recovery (motiviert durch einen realen Korruptionsfall im Produktivbetrieb)
- **H2-Upgrade auf 2.4.240**: Aktuellere DB-Engine mit verbessertem Error-Handling
- **Aufgeräumter Build**: Konsolidierte Maven-Shade-Konfiguration mit korrektem License/Notice-Handling
- **Mehrsprachigkeit (i18n)**: Komplette Präsentationsschicht in neun Sprachen (de, en, fr, it, es, pt, tr, pl, uk), erweiterbar ohne Neukompilieren über einen externen `lang/`-Ordner; eingebettete DejaVu-Unicode-Schrift für PDF-Berichte inkl. Kyrillisch (siehe Abschnitt 5.8)

Wer den exakten Stand zum Zeitpunkt der Diplomprüfung begutachten möchte, findet diesen unter dem Git-Tag `V1` im GitHub-Repository unter https://github.com/matthili/SignalReport/re

9 Installation und Betrieb

SignalReport ist als einzelne ausführbare JAR-Datei konzipiert und benötigt lediglich eine Java-21-Laufzeitumgebung. Dieses Kapitel beschreibt die verschiedenen Installationswege und den Betrieb als Hintergrund-Dienst.

9.1 Voraussetzungen

- **Java 21+** – OpenJDK (Temurin) oder Oracle JDK
- **Betriebssystem:** Windows 10/11, macOS 13+, Linux (systemd), Synology DSM 7+
- **Arbeitsspeicher:** mind. 256 MB frei (empfohlen: 512 MB)
- **Speicherplatz:** ca. 50 MB für die Anwendung; Datenbank wächst mit der Laufzeit (ca. 1 MB pro Tag bei 10-Sekunden-Intervall)
- **Netzwerk:** Port 4567 (TCP) muss frei sein

9.2 Schnellstart (portabel)

Der einfachste Weg, SignalReport zu starten:

Listing 8: Portabler Start

```
java -jar signalreport.jar
```

Anschließend ist das Web-Interface unter `http://localhost:4567` erreichbar. Beim ersten Start wird automatisch der Setup-Assistent angezeigt.

Hinweis: Bei dieser Variante läuft SignalReport nur, solange das Terminal-Fenster geöffnet bleibt. Für den Dauerbetrieb empfiehlt sich die Installation als Dienst.

9.3 Installation als Windows-Dienst

Das Projekt enthält ein Installations-Skript unter `deployment/windows/install.bat`, das SignalReport mittels Apache Commons Daemon (procrun) als Windows-Dienst einrichtet.

9.3.1 Ablauf

1. Skript als Administrator ausführen (Rechtsklick → "Als Administrator ausführen")
2. Automatische Prüfung: Administrator-Rechte und Java-Installation
3. Erstellung der Verzeichnisse:
   - `%ProgramFiles%\SignalReport` – Programmdateien
   - `%ProgramData%\SignalReport` – Daten und Logs
4. Download von Apache Commons Daemon 1.5.1 (`prunsrv.exe`)
5. Registrierung des Windows-Dienstes "SignalReport" (Autostart, LocalSystem)
6. Erstellung einer Desktop-Verknüpfung
7. Einrichtung einer Firewall-Regel für Port 4567

9.3.2 Verwaltung

Listing 9: Windows-Dienst verwalten

```
# Status pruefen
sc query SignalReport

# Dienst stoppen/starten
net stop SignalReport
net start SignalReport

# Service-Manager oeffnen (GUI)
"%ProgramFiles%\SignalReport\prunmgr.exe"
```

Die Deinstallation erfolgt über `deployment/windows/uninstall.bat` (ebenfalls als Administrator).

9.4 Installation unter Linux (systemd)

Das Skript `deployment/macos-linux/install.sh` erkennt automatisch die Plattform und erstellt unter Linux einen systemd-Service.

9.4.1 Ablauf

1. Ausführung mit `sudo bash install.sh`
2. Erstellung eines System-Benutzers `signalreport` (kein Login)
3. Verzeichnisstruktur:

- `/opt/signalreport/` – Programmdateien
- `/var/lib/signalreport/` – Datenbank und Konfiguration
- `/var/log/signalreport/` – Log-Dateien

4. Optional: Kompilierung von jsvc (Apache Commons Daemon) für professionellen Daemon-Modus; Fallback auf direkten Java-Start
5. Erstellung und Aktivierung der systemd-Unit `signalreport.service`

9.4.2 Verwaltung

Listing 10: systemd-Dienst verwalten

```
# Status pruefen
sudo systemctl status signalreport

# Dienst stoppen/starten/neustarten
sudo systemctl stop signalreport
sudo systemctl start signalreport
sudo systemctl restart signalreport

# Logs anzeigen
sudo journalctl -u signalreport -f
```

9.5 Installation unter macOS (launchd)

Das gleiche Skript (`install.sh`) erkennt macOS und erstellt einen launchd-Service.

9.5.1 Unterschiede zu Linux

- Installationsverzeichnis: `/Applications/SignalReport/`
- Datenverzeichnis: `~/Library/Application Support/SignalReport/`
- Service-Datei: `/Library/LaunchDaemons/com.signalreport.service.plist`
- Desktop-Verknüpfung: `.webloc`-Datei auf dem Schreibtisch
- Läuft als angemeldeter Benutzer (nicht als separater Systembenutzer)

9.5.2 Verwaltung

Listing 11: launchd-Dienst verwalten

```
# Dienst stoppen
sudo launchctl stop com.signalreport.service

# Dienst starten
sudo launchctl start com.signalreport.service

# Dienst komplett entladen/laden
```

```
sudo launchctl unload /Library/LaunchDaemons/com.signalreport.service
    .plist
sudo launchctl load /Library/LaunchDaemons/com.signalreport.service.
    plist
```

9.6 Synology NAS (Docker)

Auf einem Synology NAS mit DSM 7+ und installiertem Container Manager kann SignalReport als Docker-Container betrieben werden. Ein Dockerfile ist im Verzeichnis `deployment/docker/` vorbereitet.

9.7 Konfiguration

Nach dem ersten Start wird automatisch eine `config.json` im Arbeitsverzeichnis erstellt. Alle Einstellungen können über das Web-Interface geändert werden; alternativ kann die Datei manuell bearbeitet werden (bei gestopptem Dienst).

Listing 12: Beispiel config.json (Auszug)

```
{
  "measurement": {
    "intervalSeconds": 30,
    "targets": {
      "ping": "8.8.8.8",
      "dns": "google.com",
      "http": "https://example.com"
    }
  },
  "maintenanceWindow": {
    "enabled": true,
    "startHour": 4, "startMinute": 0,
    "endHour": 4, "endMinute": 10
  },
  "userInfo": {
    "provider": "Magenta",
    "customerId": "08154711",
    "userName": "Max Mustermann"
  },
  "theme": {
    "darkMode": false
  }
}
```

9.8 Backup und Migration

Für ein vollständiges Backup genügt es, zwei Dateien/Verzeichnisse zu sichern:

1. `config.json` – Alle Einstellungen (Ziele, Passwort-Hashes, Theme)
2. `data/` – H2-Datenbank mit allen Messdaten

Zur Migration auf ein anderes System: SignalReport installieren, diese beiden Dateien/Verzeichnisse kopieren und den Dienst starten. Die Datenbank und Konfiguration werden automatisch erkannt.

10 REST-API-Referenz

SignalReport stellt eine REST-API auf Port 4567 bereit, die sowohl vom integrierten Web-Interface als auch von externen Tools genutzt werden kann. Alle Antworten verwenden das JSON-Format.

10.1 Authentifizierung

Wenn die Authentifizierung aktiviert ist, erfolgt der Login über ein Challenge-Response-Verfahren. Alle nachfolgenden API-Aufrufe werden per Session-Cookie (`SR_SESSION`) authentifiziert.

Listing 13: Beispiel: Login via Challenge-Response

```
# 1. Nonce holen
curl http://localhost:4567/api/auth/nonce
# -> {"nonce":"a1b2c3..."}

# 2. Login (challengeResponse = SHA-256(SHA-256(passwort) + nonce))
curl -X POST http://localhost:4567/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","nonce":"a1b2c3...","challengeResponse":"
     d4e5f6..."}'
# -> {"token":"abc123...","role":"admin"}

# 3. Folgende Requests mit Cookie
curl -b "SR_SESSION=abc123..." http://localhost:4567/api/measurements
```

Schreibende Endpunkte (POST) erfordern Admin-Rechte (Admin-Session).

10.2 Messdaten

Methode	Pfad	Beschreibung
GET	`/api/measurements`	Letzte Messungen abrufen. Parameter: `limit` (Anzahl, Standard: 10)
GET	`/api/statistics`	Statistiken für einen Messtyp. Parameter: `type` (PING|DNS|HTTP), `hours` (Zeitraum)
GET	`/api/hourly-averages`	Stündliche Durchschnittswerte für Heatmap. Parameter: `type`, `days`

Tabelle 6: API-Endpunkte: Messdaten

Beispiel-Antwort (`/api/measurements?limit=1`):

```
[{
  "timestamp": "2026-03-15T14:32:05Z",
  "target": "8.8.8.8",
  "latencyMs": 23.45,
  "success": true,
  "type": "PING",
  "localIPv4": "192.168.1.100",
  "externalIPv4": "85.182.xxx.xxx",
  "hostHash": "a3f8b2c1..."
}]
```

10.3 Verfügbarkeit und Störungs-Lokalisierung

Diese Endpunkte werden serverseitig von `ReliabilityRoutes` (Paket `web.api`) registriert; Pfade und Verhalten sind gegenüber der monolithischen Variante unverändert.

Methode	Pfad	Beschreibung
GET	`/api/connectivity`	Störungs-Lokalisierung: vergleicht nahen/fernen Gateway mit dem Internet-Ping. Parameter: `hours`. Liefert `verdict` (Router/Gateway/Internet) und `segments`
GET	`/api/reliability`	Lückenbewusste Kennzahlen. Parameter: `hours`. Liefert Verfügbarkeit, Abdeckung, MTBF, MTTR und die aggregierte Ausfallliste
POST	`/api/outages/exclude`	Ausfallfenster aus der Wertung nehmen bzw. wieder aufnehmen. Body: `{fromEpoch, toEpoch, excluded}` (setzt die DB-Spalte `excluded`)

Tabelle 7: API-Endpunkte: Verfügbarkeit und Störungs-Lokalisierung

10.4 Dienst-Erreichbarkeit

Diese Endpunkte werden von `ServiceReachabilityRoutes` (Paket `web.api`) registriert. Das Feature ist standardmäßig deaktiviert; ist es aus, liefert `/api/services` eine leere Dienstliste mit `enabled=false`.

10.5 Berichte und Export

10.6 Daten-Aufbereitung

10.7 DNS-Benchmark

Der DNS-Benchmark vergleicht die Antwortzeiten der mitgelieferten DNS-Server (über 50 weltweit) und der eigenen Server durch parallele Abfragen mittels Virtual Threads. Ein 30-Sekunden-Cooldown verhindert übermäßige Belastung.

Methode	Pfad	Beschreibung
GET	`/api/services`	Aktueller Status je aktivem Dienst (Verdikt, „seit wann“, Erreichbarkeitsquote, letzte Methode) für Ampel/Kacheln. Parameter: `days` (Zeitfenster, Standard 30)
GET	`/api/services/history`	Episoden (Zustandswechsel) eines Dienstes – die Sperr-Timeline. Parameter: `id` (Dienst), `days` (Standard 90)
GET	`/api/services/settings`	Einstellungen abrufen: Hauptschalter, Intervall, Kontroll-SNI, vollständige Dienstliste
POST	`/api/services/settings`	Einstellungen speichern. Body: `{enabled, intervalMinutes, useControlSni, services:[{id, enabled}]}`
POST	`/api/services/check-now`	Sofortige Prüfung auslösen. Liefert `cooldownRemainingSeconds` (0 = gestartet; 5-Minuten-Abkühlphase)

Tabelle 8: API-Endpunkte: Dienst-Erreichbarkeit

10.8 IP-Tracking und Host-Info

SignalReport protokolliert automatisch Änderungen der lokalen und externen IP-Adressen. Über das Host-Hash-System können mehrere Installationen unterschieden werden, ohne personenbezogene Daten zu speichern.

10.9 Konfiguration

Alle Einstellungen (Messziele, Intervall, Wartungsfenster, Theme) werden über die Konfigurations-Endpunkte verwaltet und in `config.json` persistiert. Änderungen werden sofort wirksam, ohne Neustart.

10.10 Authentifizierung und Setup

Diese Endpunkte steuern die Ersteinrichtung (Setup-Wizard) sowie das Aktivieren, Deaktivieren und Ändern der Authentifizierung. Alle sicherheitsrelevanten Aktionen erfordern eine Challenge-Response-Verifikation – auch das Deaktivieren der Authentifizierung.

10.11 Push-Benachrichtigungen

Browser-Push-Benachrichtigungen informieren den Benutzer automatisch, wenn die Verbindungsqualität unter einen konfigurierbaren Schwellwert fällt oder wiederholt Messungen fehlschlagen.

10.12 HTTP-Statuscodes

Methode	Pfad	Beschreibung
GET	`/api/report`	PDF-Bericht generieren. Parameter: `hours` (24, 168, 8760). Antwort: PDF-Datei (application/pdf)
GET	`/api/export/csv`	CSV-Export der Rohmessungen, zeilenweise gestreamt. Parameter: `hours` (Zeitfilter), `all=true` (alle Daten), `type` (Messtyp-Filter). Antwort: CSV-Datei (text/csv, Semikolon-getrennt); bei `all=true` ein ZIP (application/zip) mit genau einer CSV-Datei
GET	`/api/export/hourly-csv`	Export der Stundenwerte (eine Zeile je Stunde, Messtyp und Ziel: Anzahl, erfolgreich, ausgenommen, Min, Durchschnitt, Median, P95, Max, Max-Zeitpunkt, Jitter). Antwort: CSV-Datei (text/csv)

Tabelle 9: API-Endpunkte: Berichte und Export

Methode	Pfad	Beschreibung
GET	`/api/dataprep/status`	Konfiguration und Zustand der Daten-Aufbereitung: letzter Lauf (Beginn/Ende), verdichtete Stunden, gelöschte Rohzeilen, letzter Fehler, Stand der Stundenwerte, Anzahl Roh- und Stundenzeilen, ausstehende Stunden
POST	`/api/dataprep/run-now`	Startet den vollen Lauf (Rückstand verdichten, Aufbewahrung anwenden) sofort im Hintergrund. Antwort: `{started, cooldownRemainingSeconds}`; Abkühlphase 5 Minuten

Tabelle 10: API-Endpunkte: Daten-Aufbereitung

11 Anhang A: UML-Diagramme (vollständig)

Dieser Anhang enthält alle vollständigen UML-Diagramme in hoher Auflösung.

11.1 Klassen-Diagramm

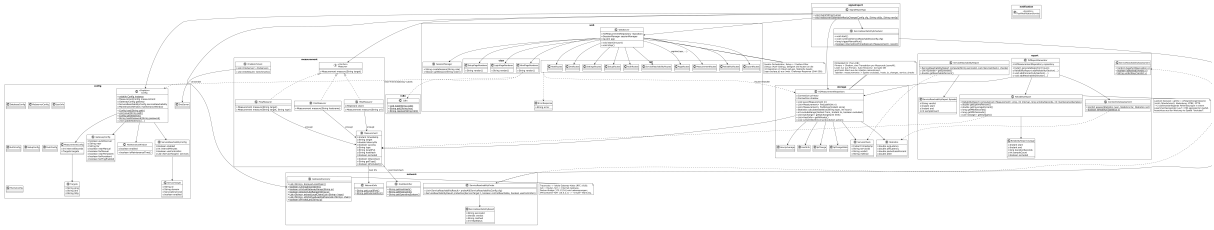


Abbildung 8: Vollständiges Klassen-Diagramm

Methode	Pfad	Beschreibung
GET	`/api/dns/servers`	Server eines vollen Benchmarks: mitgelieferte ohne abgewählte, dazu eingeschaltete eigene (Feld `custom`)
GET	`/api/dns/catalog`	Mitgelieferte Liste mit Abwahl-Status, eigene Server, Obergrenze für eigene Server
POST	`/api/dns/settings`	Eigene Server und abgewählte mitgelieferte speichern. Body: `customServers` (Name, IP-Adresse, Notiz, `enabled`), `hiddenBuiltins` (Adressen). Ungültige Einträge (Hostname, Dublette, bereits mitgeliefert, mehr als 50): 400 mit lesbarer Meldung
POST	`/api/dns/benchmark`	DNS-Benchmark starten. Parameter: `hostname`, `scope` (`all` = voller Lauf, `custom` = alle eigenen Server). Fragt die Server parallel ab (Virtual Threads). 30-Sekunden-Cooldown.
GET	`/api/dns/statistics`	DNS-Statistik (Anzahl Server, davon eigene, Regionen und Provider der mitgelieferten)

Tabelle 11: API-Endpunkte: DNS-Benchmark

Methode	Pfad	Beschreibung
GET	`/api/ip-changes`	Letzte IP-Änderungen. Parameter: `limit` (Standard: 50)
GET	`/api/ip-statistics`	Statistik über IP-Änderungen pro Host (Anzahl, erster/letzter Wechsel)
GET	`/api/hosts`	Liste aller registrierten Hosts
GET	`/api/host/current`	Aktueller Host: Hostname, Hash, IPv4/IPv6

Tabelle 12: API-Endpunkte: IP-Tracking

11.2 Komponenten-Diagramm

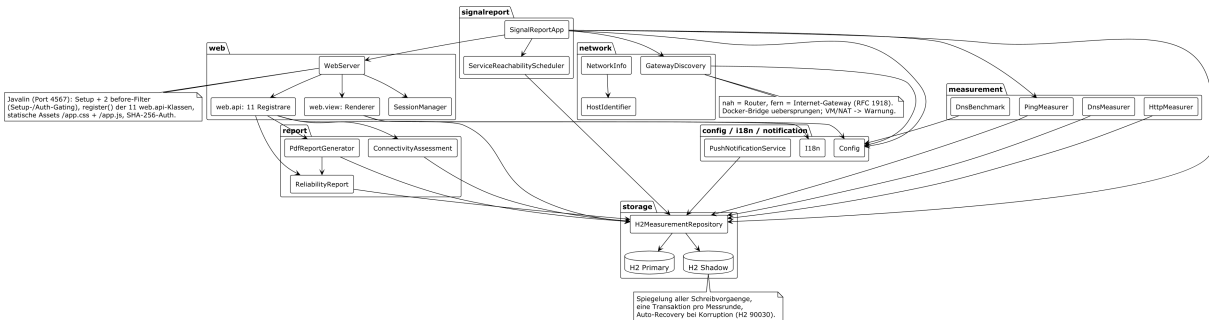


Abbildung 9: Vollständiges Komponenten-Diagramm

Methode	Pfad	Beschreibung
GET	`/api/config/current`	Aktuelle Konfiguration abrufen (Messziele, Intervall, Maintenance, Benutzer-Info, Gateway-Objekt inkl. `virtualSuspected`, Sprache)
POST	`/api/config/update`	Konfiguration aktualisieren. Body: JSON mit ping, dns, http, intervalSeconds, maintenance, userInfo sowie `gateway` (nearManual, near, nearPersistent, farManual, far, farPersistent, farPingEnabled)
POST	`/api/config/language`	UI-Sprache umstellen. Body: `{language}` (ISO-Code)
GET	`/api/theme/settings`	Theme-Einstellungen abrufen
POST	`/api/theme/settings`	Theme ändern. Body: `{``darkMode'': true}`

Tabelle 13: API-Endpunkte: Konfiguration

11.3 Sequenz-Diagramm: Messungsablauf

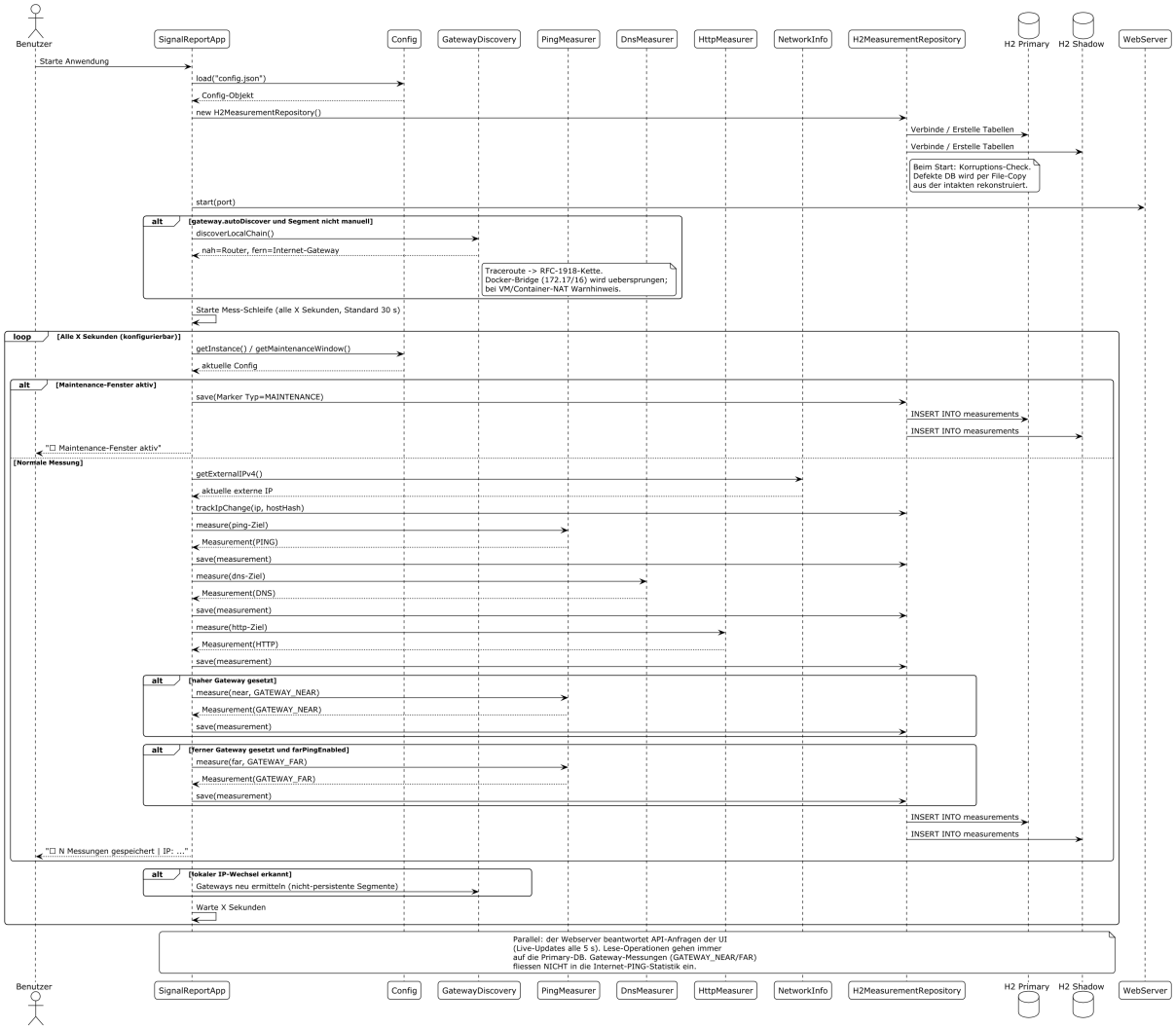


Abbildung 10: Vollständiges Sequenz-Diagramm Messungsablauf

Methode	Pfad	Beschreibung
GET	`/api/auth/nonce`	Einmal-Nonce für Challenge-Response generieren (60 s gültig)
POST	`/api/auth/login`	Login via Challenge-Response. Body: `{username, nonce, challengeResponse}`
POST	`/api/auth/logout`	Session beenden (Cookie wird gelöscht)
GET	`/api/auth/status`	Auth-Status abfragen (aktiviert/deaktiviert)
GET	`/api/setup/status`	Status der Ersteinrichtung abfragen (abgeschlossen: ja/nein)
POST	`/api/setup/complete`	Ersteinrichtung abschließen. Body: `{adminPasswordHash, enableAuth, userPasswordHash}`
POST	`/api/auth/enable`	Auth aktivieren (Challenge-Response + User-Passwort-Hash)
POST	`/api/auth/disable`	Auth deaktivieren (Challenge-Response erforderlich)
POST	`/api/auth/change-admin`	Admin-Passwort ändern (Challenge-Response für altes PW + neuer Hash)

Tabelle 14: API-Endpunkte: Authentifizierung

Methode	Pfad	Beschreibung
GET	`/api/push/settings`	Push-Einstellungen abrufen
POST	`/api/push/settings`	Push-Einstellungen ändern (Schwellwert, Anzahl)

Tabelle 15: API-Endpunkte: Push-Benachrichtigungen

Code	Bedeutung
200	Erfolgreiche Anfrage
302	Weiterleitung (z. B. zum Setup)
401	Authentifizierung erforderlich oder fehlgeschlagen
403	Keine Berechtigung (z. B. User versucht Admin-Aktion)
500	Interner Serverfehler (Details im Response-Body)

Tabelle 16: Verwendete HTTP-Statuscodes

11.4 Sequenz-Diagramm: PDF-Export

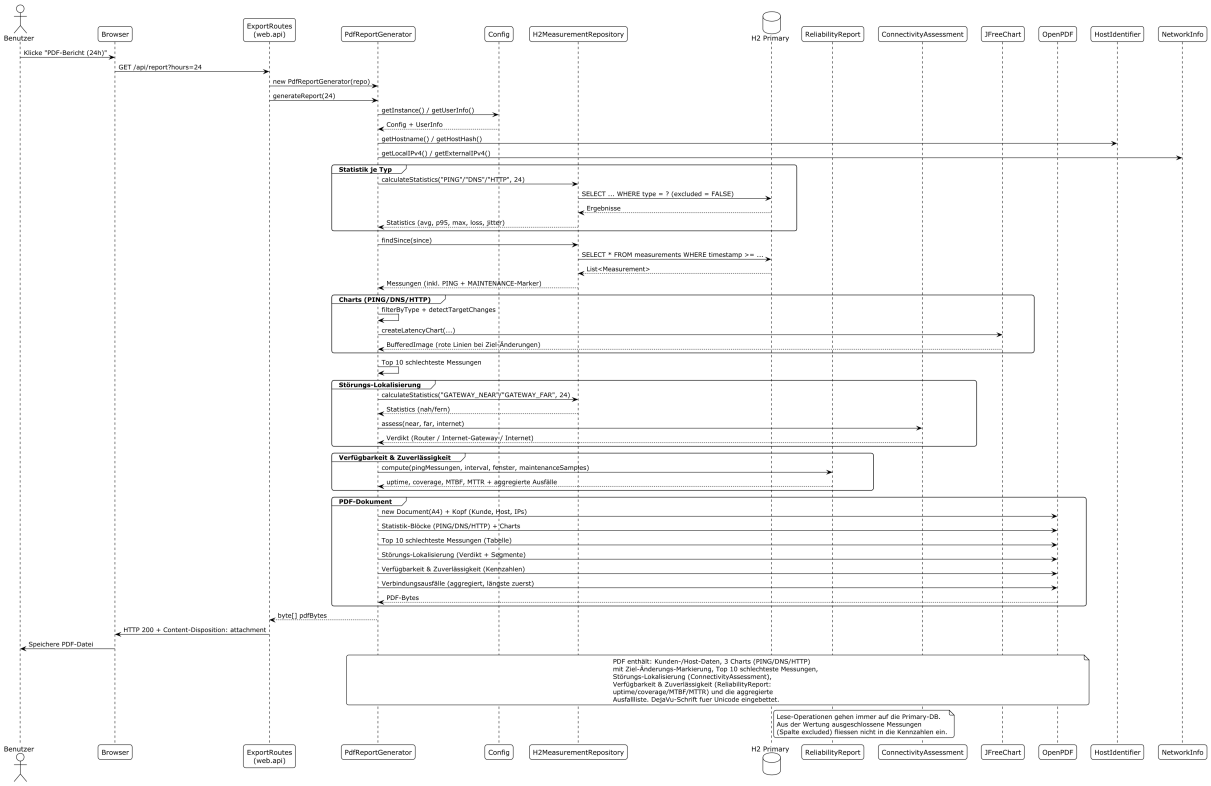


Abbildung 11: Vollständiges Sequenz-Diagramm PDF-Export

11.5 Use-Case-Diagramm

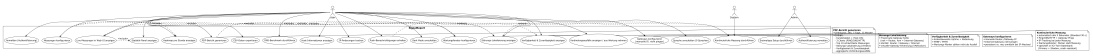


Abbildung 12: Vollständiges Use-Case-Diagramm

11.6 Deployment-Diagramm

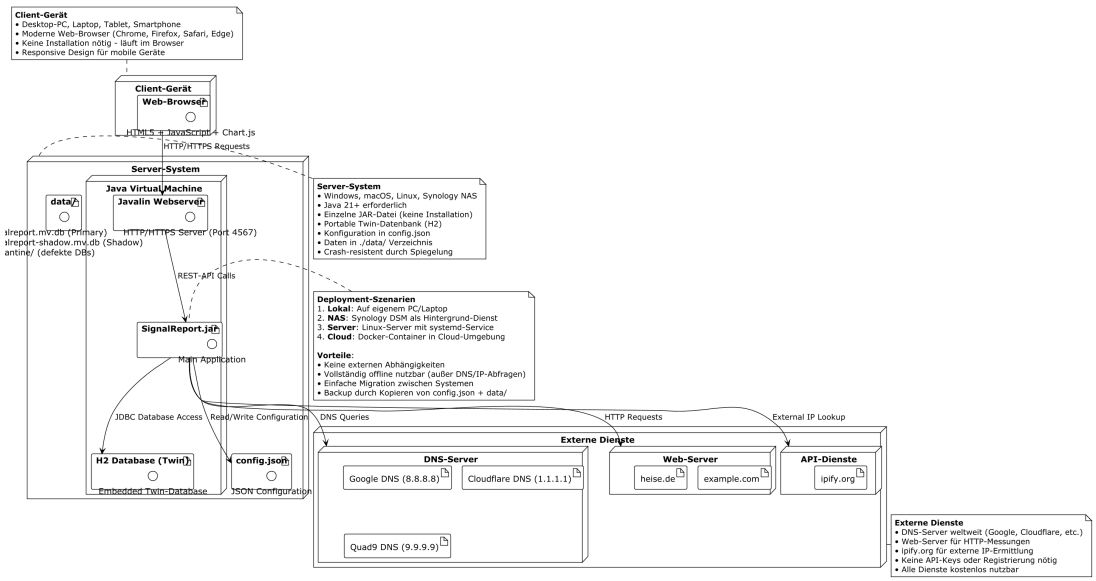


Abbildung 13: Vollständiges Deployment-Diagramm

11.7 State-Diagramm: Authentifizierung

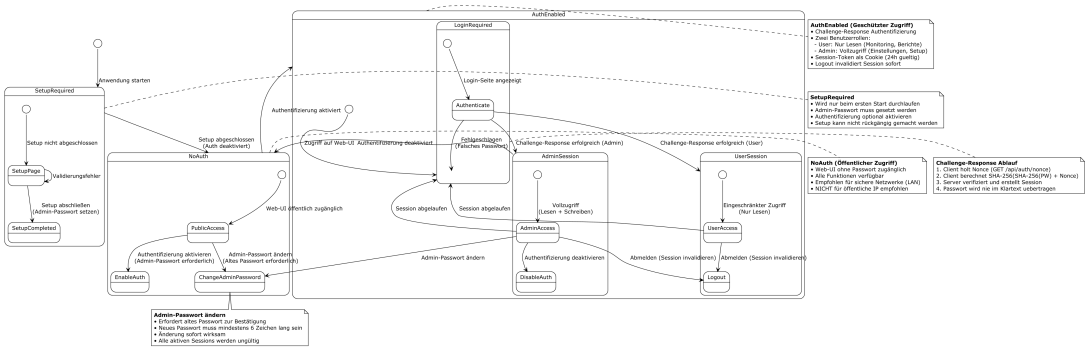


Abbildung 14: Vollständiges State-Diagramm Authentifizierung